Zellic

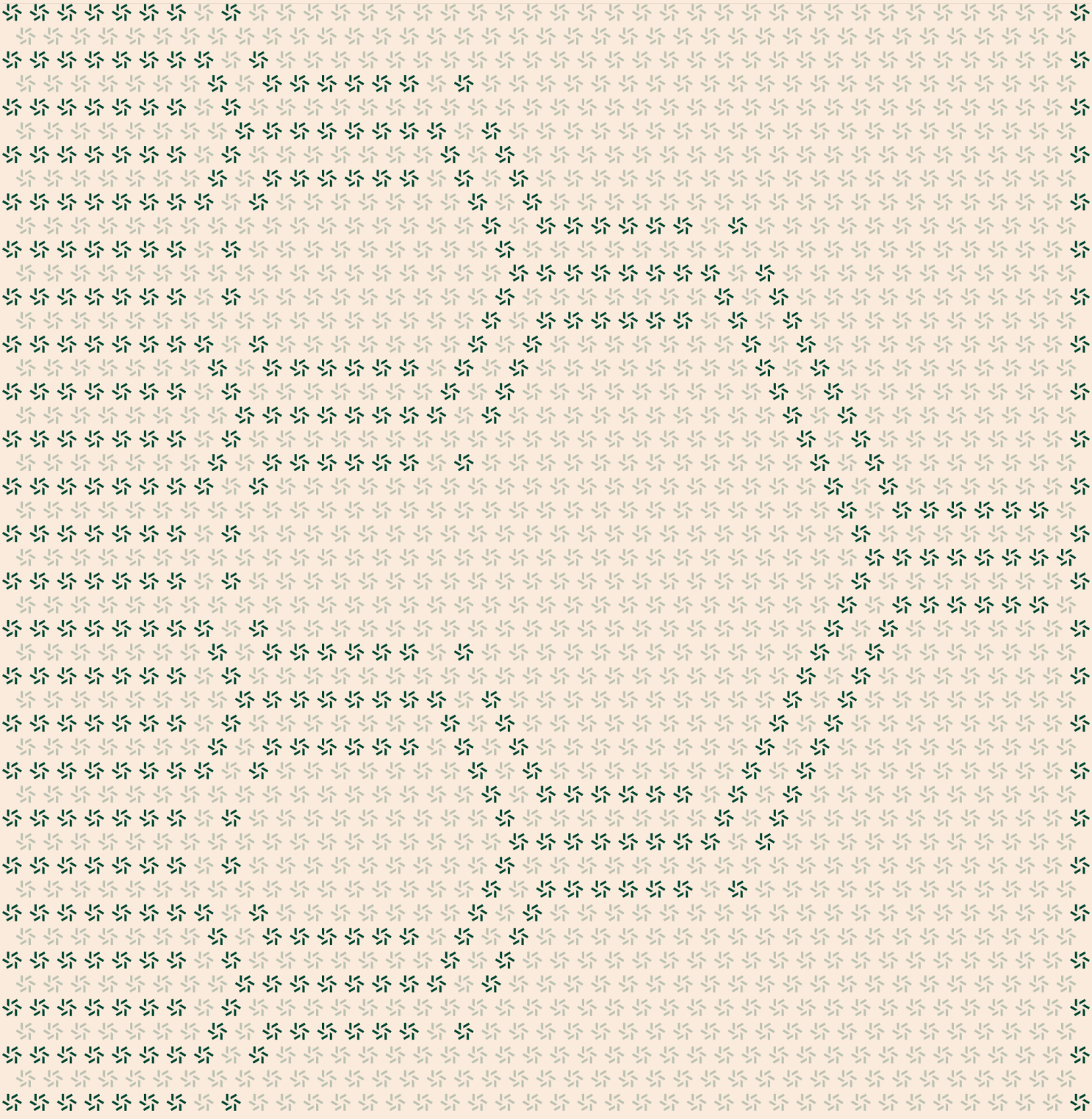
July 24, 2026

Prepared for
Karol Podufalski
Jonathan Kelsey
Andrija Novakovic
Nori

Prepared by
Katerina Belotskaia
Jiaxin Guan
Hyunsik Jeong
Jaeeu Kim
Jinseo Kim
Malte Leip
Mark Pedron
Zellic

Nori Bridge

Application Security Assessment



Contents

About Zellic **5**

1. Overview **6**

1.1. Executive Summary 6

1.2. Goals of the Assessment 6

1.3. Non-goals and Limitations 6

1.4. Results 7

2. Introduction **9**

2.1. About Nori Bridge 9

2.2. Methodology 9

2.3. Scope 11

2.4. Project Overview 12

2.5. Project Timeline 13

3. Detailed Findings **15**

3.1. [b1114] Aggregation leaf circuit `layer1` does not constrain disabled proofs to act as identity, allowing forgery of the SP1 PLONK public inputs 15

3.2. [18fa3] SP1 public value `vk_root` is not pinned, allowing forgery of public values 21

3.3. [ec50d] Missing on-curve and subgroup checks in the Groth16 recursion verifier 28

3.4. [d3034] Compromised admin is not prevented from minting or unlocking illegitimately 33

3.5. [4279a] Unvalidated `oldestAction` in `dispatchAndEvict` can permanently brick ETH to Mina minting 38

3.6. [a3850] The committed deposit root is not constrained to be complete 42

3.7. [1eb72] Missing in-circuit head-advancement and checkpoint-slot assertions 47

3.8. [a2090] Merkle zero index reversed 51

3.9. [41428] In-flight mints are invalidated by the next `update` 55

3.10. [8ff57] Empty updates response causes `get_first_update` to panic 58

3.11. [7f3a1] Mempool-visible `codeChallenge` allows griefing of `lockTokens` callers 61

3.12. [1f602] Function `wordToBytes` does not guarantee canonicity for large `bytesPerWord` 63

3.13. [b165f] Usage of `Provable.switch` without an explicit range constraint on `shift_power` 66

3.14. [0dd9c] Function `ArrayListHasher.hash` does not enforce configured array length 68

3.15. [10b08] Parallel implementations of shared utilities across Groth16 and PLONK components 71

3.16. [1a697] Function `Field.toBigInt` is a debug-only o1js operation but is used outside debug contexts 78

3.17. [a9dea] Undocumented value-dependence for circuit-generating functions 80

3.18. [adcd3] Groth16 proof parser does not validate that `piN` keys are sequential 82

3.19. [af97e] Usage of deprecated `Gadgets.SHA256` API 85

3.20. [d7ffa] Usage of `@internal` o1js APIs 88

3.21. [b8891] Constructor `AffineCache` assigns `yp_prime` twice 91

3.22. [e68c5] Function `G2Line.evaluate_g1` is dead code and does not implement the correct evaluation of the line function on G_1 93

3.23. [e4e27] Function `prepare_consensus_mpt_proof_inputs` reconstructs `ExecutionHttpProxy` from env on every window 94

4. Discussion 96

4.1. Use of a pre-release o1js build (3.0.0-mesa.698ca) across the Nori codebases 96

4.2. Unused functions and constants in proof-conversion 97

4.3. Field element `w27` lacks a canonical source and documentation of its derivation 98

4.4. Difference in the KZG folding randomizer derivation between proof-conversion and the SP1 Rust verifier 100

5. Threat Model 103

5.1. Module: MinaStateSettlement.sol 103

5.2. Module: NoriTokenBridge.sol 105

6. Assessment Results 111

6.1. Disclaimer 112

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Nori from May 11th to June 22nd, 2026. During this engagement, Zellic reviewed Nori Bridge's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are the proof-conversion verification and aggregation circuits constrained correctly, with no underconstrained or unbound public input that would let an invalid proof be accepted?
- Do the Rust `nori-program` SP1 guest program and the o1js Mina smart contract implementations agree on the proof-output encoding, the Poseidon leaf hashing, and the Merkle-root construction?
- Can the consensus state transition or the Helios store-hash chain be manipulated to settle a forged state or deposit root?
- Can tokens be minted on Mina without a valid Ethereum deposit, minted more than once, or minted by anyone other than the deposit's intended recipient?
- Can Ethereum be unlocked on Ethereum without a valid burn of corresponding tokens on Mina, or be unlocked by anyone other than the intended recipient?
- Are privileged operations properly access-controlled? Can a malicious actor brick or stall the bridge or lock user funds?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end and client-side components
- Off-chain infrastructure, including the operators, provers, and RPC endpoints
- Custody of the privileged keys
- The upstream `sp1-helios` light client that `nori-bridge-head` is forked from
- The `pairing-utils` Rust library bundled with `proof-conversion`
- Usage of proof-conversion to convert proofs other than those produced by SP1 v6.1.0
- The internal cryptographic soundness of the underlying proof systems the bridge builds on, namely the SP1 zkVM and the proof system underlying o1js
- Security or configuration of the external Aligned Layer dependency

- Suitability of the current code for, or potential design of, future changes, such as for migrations on Helios updates or RISC0 integration

Our audit of nori-program/src/consensus.rs and nori-program/src/mpt.rs in the nori-bridge-head repository is based on their diff with the Helios implementation from this SP1 Helios commit ↗. We assume correct implementations of the Helios logic in `program/src/light_client.rs` and `primitives/src/lib.rs` of the SP1 Helios repository, and only evaluate the changes made in the `nori-bridge-head` repository compared to the SP1 Helios implementation.

The code utilizing o1js depended on, at the time of audit, experimental o1js 3.0.0 pre-release versions, for which documentation was not available. Our audit was based on the established documented behavior of the latest stable o1js version. For more details see section 4.1. ↗.

The PLONK verifier implementation in proof-conversion was audited against the SP1 v6.1.0 Rust verifier (`sp1/crates/verifier/src/plonk` ↗). We thus audited the circuits in src/plonk/recursion/ and their dependencies with regards to the relationship between knowing compatible proofs for the `ZkPrograms` and knowing an SP1 PLONK proof that is accepted by the Rust SP1 PLONK verifier, rather than checking whether that PLONK proving system is itself cryptographically sound.

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide. Unlike a comprehensive audit, this format does not aim for complete coverage of the codebase, and we do not guarantee that all vulnerabilities will be identified.

1.4. Results

During our assessment on the scoped Nori Bridge circuits, we discovered 23 findings. Three critical issues were found. Three were of high impact, one was of medium impact, four were of low impact, and the remaining findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Nori in the Discussion section (4. ↗).

Based on the number and severity of findings identified during this engagement, we recommend a comprehensive reassessment of the codebase following remediation efforts; see section 6. ↗

Breakdown of Finding Impacts

Impact Level	Count
Critical	3
High	3
Medium	1
Low	4
Informational	12



2. Introduction

2.1. About Nori Bridge

Nori contributed the following description of Nori Bridge:

> Nori is a trust-minimized bridge between Ethereum and Mina whose purpose is to make the cryptographically verified state of each chain available to applications on the other and consists of five components:
>
> 1. **Ethereum `NoriTokenBridge`** - accepts user ETH locks, binds each lock to a user-supplied `codeChallenge`, and emits deposit-root state.
> 2. **`nori-bridge-head` (Rust)** - runs the Helios consensus light client inside the SP1 zkVM, producing an SP1-Plonk proof of Ethereum consensus advancement together with the MPT inclusion of the bridge contract's deposit-root state.
> 3. **`proof-conversion`** - converts (verifies) the SP1/Plonk proof into an o1js-compatible proof that can be verified on Mina, using Geometry Research's `o1js-blobstream` for the underlying verification.
> 4. **Mina `NoriTokenBridge` zkApp** - verifies the converted proof inside `update`, advances the on-chain checkpoint, and authorizes minting of `nETH` via the `FungibleToken` contract and a per-user `NoriStorageInterface`.
> 5. **Mina→Ethereum return path** - using AlignedLayer to verify Mina consensus proofs on Ethereum. This inherits AlignedLayer's crypto-economic trust assumptions, which is a different (and weaker) trust model than the cryptographic guarantee of the forward path. Built on top of audited `mina_bridge` by lambda class.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Underconstrained circuits. The most common type of vulnerability in a ZKP circuit is not adding sufficient constraints to the system. This leads to proofs generated with incorrect witnesses in terms of the specification of the project being accepted by the ZKP verifier. We manually check that the set of constraints satisfies soundness, enough to remove all such possibilities and in some cases provide a proof of the fact.

Overconstrained circuit. While rare, it is possible that a circuit is overconstrained. In this case, appropriately assigning witnesses will become impossible, leading to a vulnerability. To prevent this, we manually check that the constraint system is set up with completeness so

that the proofs generated with the correct set of witnesses indeed pass the ZKP verification.

Missing range checks. This is a popular type of an underconstrained-circuit vulnerability. Due to the usage of field arithmetic, overflow checks and range checks serve a huge purpose to build applications that work over the integers. We manually check the code for such missing checks and, in certain cases, provide a proof that the given set of range checks is sufficient to constrain the circuit up to specification.

Cryptography. ZKP technology and their applications are based on various aspects of cryptography. We manually review the cryptography usage of the project and examine the relevant studies and standards for any inconsistencies or vulnerabilities.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices, guidelines, and code quality standards.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped circuits itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets[1]:

Nori Bridge Circuits

Types	TypeScript, Rust, Solidity
Platforms	Mina, EVM-compatible
Target	proof-conversion
Repository	https://github.com/Nori-zk/proof-conversion ↗
Version	`05d307b7659ce10d84809b914b28628ad4bfc4ba`

Programs	`plonk/verifier.ts`
	`plonk/fiat-shamir/*`
	`plonk/piop/*`
	`plonk/mm_loop/*`
	`plonk/recursion/*`
	`groth/verifier.ts`
	`groth/recursion/*`
	`towers/*`
	`node_resolver.ts`
	`sha/utils.ts`
	`compressor/compressor.ts`

[1] Excluding all auto-generated files, tests, mocks, and interfaces in all components.

Target	nori-bridge-sdk
Repository	https://github.com/Nori-zk/nori-bridge-sdk ↗
Version	`979d67cc0e8121a37c3dcc3274745065c84d3857`

Programs	`contracts/ethereum/contracts/NoriTokenBridge.sol`
	`contracts/ethereum/contracts/MinaStateSettlement.sol`
	`contracts/ethereum/contracts/MinaAccountValidation.sol`
	`contracts/mina/src/NoriTokenBridge.ts`
	`contracts/mina/src/NoriStorageInterface.ts`
	`contracts/mina/src/depositAttestation.ts`
	`o1js-zk-utils/src/merkle-attestor/merkleTree.ts`
	`o1js-zk-utils/src/types.ts`
	`o1js-zk-utils/src/utils.ts`

Target	nori-bridge-head
Repository	https://github.com/Nori-zk/nori-bridge-head ↗
Version	`30db225a880ff4dc5c336ec3f4de2d20e2f21d52`

Programs	`nori-hash/src/helios.rs`
	`nori-hash/src/merkle_poseidon_fixed.rs`
	`nori-primitives/src/types.rs`
	`nori-program/src/consensus.rs`
	`nori-program/src/mpt.rs`
	`nori/src/rpcs/consensus/mod.rs`
	`nori/src/rpcs/execution/http.rs`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 6.25 person-weeks. The assessment was conducted by seven consultants over the course of six calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
Engagement Manager
pedro@zellic.io

The following consultants were engaged to conduct the assessment:

Katerina Belotskaia
Engineer
kate@zellic.io

Jiaxin Guan
Engineer
jiaxin@zellic.io

Hyunsik Jeong
Engineer
hyunsik@zellic.io

Jaeeu Kim
Engineer
jaeeu@zellic.io

Jinseo Kim
Engineer
jinseo@zellic.io

Malte Leip
Engineer
malte@zellic.io

Mark Pedron
Engineer
mark.pedron@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

May 11, 2026	Kick-off call
May 11, 2026	Start of primary review period
June 22, 2026	End of primary review period

3. Detailed Findings

3.1. [b1114] Aggregation leaf circuit `layer1` does not constrain disabled proofs to act as identity, allowing forgery of the SP1 PLONK public inputs

Target	proof-conversion::compressor::layer1		
Category	Soundness	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

The purpose of the circuits in `src/compressor` in the `proof-conversion` repository is to wrap the PLONK and Groth16 proof verifier circuits, which are split up into several proofs each due to their complexity, as a binary tree of `ZkProgram` recursion proofs, resulting in a single final proof. The leaf layer of the tree consists of N sequential proof steps `zkp0`, ..., `zkp(N-1)` (with $N = 24$ for SP1 PLONK), each carrying a `Field`-valued Poseidon-based accumulator forward via its `publicInput` / `publicOutput`: `zkp(i+1).publicInput` is required to equal `zkp(i).publicOutput`, so that the sequence forms a chain from `zkp0.publicInput` to `zkp(N-1).publicOutput`. Above the leaves, the `layer1` recursion verifier (defined in `src/compressor/layer1node.ts`) combines two leaf proofs into a `SubtreeCarry`, and the `node` recursion verifier (`src/compressor/compressor.ts`) combines two such subtrees into a larger `SubtreeCarry`. A `SubtreeCarry` summarizes a contiguous segment of the chain via three `Field`s: `leftIn` is the leftmost leaf's `publicInput`, `rightOut` is the rightmost leaf's `publicOutput`, and `subtreeVkDigest` is a Poseidon commitment to the verification keys of every leaf and every internal `layer1`/`node` recursion verifier of the subtree.

The consumer of the resulting root proof (for Nori using SP1 PLONK, this is the `ethVerify` function of the `NoriTokenBridge` Mina smart contract in the `nori-bridge-sdk` repository) pins the root proof's `subtreeVkDigest` against a constant that was precomputed offline from the legitimate leaf and recursion verification keys. The consumer additionally interprets the root proof's `rightOut` as the Poseidon hash of the underlying SP1 PLONK proof's public values and checks it against the user-supplied claimed public values.

To allow N to not be a power of two, the leaf layer is padded out with dummy leaves up to the next power of two, and the corresponding `layer1` proofs may have one or both of their verifications disabled. Pinning the root `subtreeVkDigest` is what guarantees that a malicious prover cannot disable verification of legitimate leaves: the precomputed root digest commits, layer by layer, to which positions were intended to be disabled (the padding positions get `NOTHING_UP_MY_SLEEVE = 0` in place of the leaf vk hash), so the attacker can only skip verification of the proofs that were already expected to be skipped.

The `layer1` recursion verifier is implemented as follows:

```
// proof-conversion: src/compressor/layer1node.ts
```

```
async method(
  piLeft: ZkpProofLeft,
  vkLeft: VerificationKey,
  verifyLeft: Bool,
  piRight: ZkpProofRight,
  vkRight: VerificationKey,
  verifyRight: Bool
) {
  piLeft.verifyIf(vkLeft, verifyLeft);
  piRight.verifyIf(vkRight, verifyRight);

  piLeft.publicOutput.assertEquals(piRight.publicInput);

  const leftVkHash = Provable.if(
    verifyLeft,
    vkLeft.hash,
    NOTHING_UP_MY_SLEEVE
  );
  const rightVkHash = Provable.if(
    verifyRight,
    vkRight.hash,
    NOTHING_UP_MY_SLEEVE
  );

  const subtreeVkDigest = Poseidon.hash([
    leftVkHash,
    rightVkHash,
    Field(1), // layer
  ]);

  return {
    publicOutput: new SubtreeCarry({
      leftIn: piLeft.publicInput,
      rightOut: piRight.publicOutput,
      subtreeVkDigest: subtreeVkDigest,
    }),
  };
}
```

The line `piLeft.verifyIf(vkLeft, verifyLeft)` verifies `piLeft` against `vkLeft` only when `verifyLeft` is true. When `verifyLeft` is false, no relationship between `piLeft.publicInput` and `piLeft.publicOutput` is enforced by `verifyIf`. The same applies to `piRight.verifyIf(vkRight, verifyRight)`.

For each disabled side, the circuit should additionally enforce that the side acts as the identity on the chain endpoint, namely, that its `publicInput` equals its `publicOutput`, so that the corresponding `SubtreeCarry` field correctly carries the chain through the dummy. The `layer1`

circuit does not enforce this. The only constraint relating the two sides' values is `piLeft.publicOutput.assertEquals(piRight.publicInput)`, which ties the "middle" value of the subtree but leaves each side's outer endpoint (`piLeft.publicInput` and `piRight.publicOutput`, populating `leftIn` and `rightOut`) unconstrained for any disabled side.

In the intended honest flow, the prover constructs padding leaves as

```
// proof-conversion/src/node_resolver.ts
return ZkpProofLeft.dummy(pLast.publicOutput, pLast.publicOutput, 0);
// ...
return ZkpProofRight.dummy(pLast.publicOutput, pLast.publicOutput, 0);
```

with the dummy's `publicInput` equal to its `publicOutput`, so that the padded subtree acts as the identity on the chain endpoint. A malicious prover is not bound by this convention.

To confirm this, the following POC verifies a `layer1` proof successfully with both sides disabled, but with `publicOutput.leftIn = 111` and `publicOutput.rightOut = 333` having non-equal values.

```
import fs from 'fs';
import { Bool, Cache, Field, VerificationKey, verify } from 'o1js';
import { ZkpProofLeft, ZkpProofRight } from '../structs.js';
import { layer1 } from './layer1node.js';

const cacheDir = process.argv[2] ?? './cache_b1114';
const prodVkPath = process.argv[3]; // optional

const A = Field(111n); // attacker-chosen leftIn
const M = Field(222n); // shared middle (satisfies piLeft.publicOutput ==
    piRight.publicInput)
const C = Field(333n); // attacker-chosen rightOut, intentionally != A

const { verificationKey: layer1Vk } = await layer1.compile({
  cache: Cache.FileSystem(cacheDir),
});

if (prodVkPath) {
  const prodVk = await VerificationKey.fromJSON(
    JSON.parse(fs.readFileSync(prodVkPath, 'utf8'))
  );
  if (prodVk.hash.toString() !== layer1Vk.hash.toString()) {
    throw new Error(
      'Compiled layer1 VK hash does not match production VK hash — ' +
        'o1js version drift or program change. PoC abort.'
    );
  }
}
```

```
// Dummy DynamicProofs with arbitrary public inputs/outputs.
const piLeft = await ZkpProofLeft.dummy(A, M, 0);
const piRight = await ZkpProofRight.dummy(M, C, 0);

// As `vkLeft` and `vkRight` won't actually be used, we can use something
    arbitrary
// of the correct type.
const unusedLeafVk = layer1Vk;

const { proof } = await layer1.compute(
  piLeft,
  unusedLeafVk,
  Bool(false),
  piRight,
  unusedLeafVk,
  Bool(false)
);

const leftIn = proof.publicOutput.leftIn.toString();
const rightOut = proof.publicOutput.rightOut.toString();
const ok = await verify(proof, layer1Vk);

if (!ok) throw new Error('PoC failed: proof did not verify');

console.log(
  `PoC SUCCESS: a valid layer1 proof was produced with ` +
    `leftIn=${leftIn} != rightOut=${rightOut}, no underlying zkps verified.`
);
```

Impact

The bug is exploitable end-to-end whenever the leaf-layer zkp count N is not a power of two. The SP1 PLONK configuration uses $N = 24$, padded to $32 = 2^5$. The 16 nodes at the `layer1` level then have the following flag values, baked into the pinned root digest:

- Nodes 0 through 11 verify both leaves (`verifyLeft = verifyRight = true`).
- Nodes 12 through 15 verify neither leaf (`verifyLeft = verifyRight = false`).

Nodes 12 through 15 are where the bug applies: at each of those four `layer1` nodes, the prover can choose `leftIn` and `rightOut` freely.

The `node` recursion verifier in `src/compressor/compressor.ts` propagates the chain across layers ≥ 2 by setting

```
// proof-conversion: src/compressor/compressor.ts
piLeft.verify(vkLeft);
piRight.verify(vkRight);

piLeft.publicOutput.rightOut.assertEquals(piRight.publicOutput.leftIn);
// ...
return {
  publicOutput: new SubtreeCarry({
    leftIn: piLeft.publicOutput.leftIn,
    rightOut: piRight.publicOutput.rightOut,
    subtreeVkDigest,
  }),
};
```

This enforces continuity between the two children's middle endpoints but propagates only `piLeft.leftIn` and `piRight.rightOut` to the parent. As a result, the chain through the four broken `layer1` nodes 12–15 on the right half of the tree is constrained only at the left end. The root proof's `rightOut` can thus be freely chosen by the malicious prover via node 15's free `rightOut`.

The Mina-side bridge consumes the converted SP1 PLONK proof in the `ethVerify` function of the `NoriTokenBridge` smart contract via

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts

proof.verify(vk);

// Passed proof matches extracted public entry 2
proof.publicOutput.subtreeVkDigest.assertEquals(ethNodeVk);

// ...

const pi0 = ethPlonkVK;
const pi1 = parsePlonkPublicInputsProvable(Bytes.from(bytes));

const piDigest = Poseidon.hashPacked(Provable.Array(FrC.provable, 2), [
    pi0,
    pi1,
]);

// ...

piDigest.assertEquals(proof.publicOutput.rightOut);
```

where `pi0` is the SP1 nori-bridge-head program verification key (a constant for the deployed bridge) and `pi1` is a hash of the user-supplied public value bytes.

Thus, the attacker's ability to freely choose `rightOut` of the root proof results in their ability to claim the `pi0` verification key as expected for the nori-bridge-head program, while claiming arbitrary incorrect public values. This allows them to cause the bridge to update for example its head, `verifiedStateRoot`, and deposit roots based on attacker-chosen values, allowing the attacker to arrange arbitrary mints for themselves, breaking the bridge's soundness.

Recommendations

In `layer1`, additionally constrain each disabled side to act as the identity on the chain endpoint. Concretely, when a side's verification flag is `false`, that side's `publicInput` should be constrained to be equal to its `publicOutput`.

Remediation

This issue has been acknowledged by Nori, and a fix was implemented in commit d9aec26b ↗.

3.2. [18fa3] SP1 public value `vk_root` is not pinned, allowing forgery of public values

Target	proof-conversion::plonk		
Category	Soundness	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

The proof-conversion repository contains circuits to verify Groth16, as well as PLONK proofs produced by SP1. We will describe this finding first in terms of the PLONK path, as this is what was intended to be used with the Mina smart contract in the audited version. The Groth16 verifier would have a similar issue if it were used, and we will discuss this at the end.

PLONK

An SP1 v6.1.0 PLONK proof exposes exactly five BN254 scalars as PLONK public inputs (see `crates/verifier/src/proof.rs:17` and `crates/recursion/gnark-ffi/go/sp1/sp1.go:29` in the SP1 repository, at tag v6.1.0):

Numbered name	Descriptive name	Description
`pi0`	`vkey_hash`	Hash of the user ELF's `MachineVerifyingKey`; intended to commit to the proved program.
`pi1`	`committed_values_digest`	Digest of bytes the guest program wrote to the program's public output.
`pi2`	`exit_code`	The exit code of the guest program.
`pi3`	`vk_root`	Merkle root of the SP1 recursion verification keys.
`pi4`	`proof_nonce`	Caller-supplied nonce, allows creating distinct proofs even for identical programs and inputs.

The `proof-conversion` o1js translation of the SP1 PLONK verifier propagates only the first two of these to consumers of the root proof. Its leaf accumulator culminates in `zkp23`, whose `publicOutput` is the digest of just `pi0` and `pi1`:

```
// proof-conversion: src/plonk/recursion/zkp23.ts
return {
  publicOutput: Poseidon.hashPacked(Provable.Array(FrC.provable, 2), [
    acc.proof.pi0,
    acc.proof.pi1,
  ]),
};
```

The Mina-side bridge consumer constrains both: `pi0` is compared against the on-chain state variable `noriHeliosProgramPi0`, and the commitment `pi1` is verified to open to user-supplied values.

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts
const pi0 = ethPlonkVK;
const pi1 = parsePlonkPublicInputsProvable(Bytes.from(bytes));

const piDigest = Poseidon.hashPacked(Provable.Array(FrC.provable, 2), [
    pi0,
```

```
    pi1,
]);

piDigest.assertEquals(proof.publicOutput.rightOut);
```

The remaining three public inputs `pi2`, `pi3`, `pi4` are not exposed via the chain's `publicOutput`, and the bridge has no way to constrain them. Inside the chain, they appear in exactly two places, both of them mechanical parts of the PLONK verification equation. In `zkp0` they are absorbed into the Fiat-Shamir challenger to derive `gamma`,

```
// proof-conversion: src/plonk/recursion/zkp0.ts
acc.fs.squeezeGamma(
  acc.proof,
  acc.state.pi0, acc.state.pi1, acc.state.pi2, acc.state.pi3, acc.state.pi4,
  VK,
);
```

and in `zkp3` they contribute to the public-input term of the PLONK proof verification:

```
// proof-conversion: src/plonk/recursion/zkp3.ts
const pis = pi_contribution(
  [acc.state.pi0, acc.state.pi1, acc.state.pi2, acc.state.pi3, acc.state.pi4],
  acc.fs.zeta,
  acc.state.zh_eval,
  VK.inv_domain_size,
  VK.omega,
);
```

Neither site compares these values against expected constants. The o1js chain therefore accepts any PLONK proof that is internally consistent with prover-chosen `pi2`, `pi3`, and `pi4`.

Groth16

The SP1 Groth16 proofs have the same five public values as the PLONK proofs. In contrast to the o1js PLONK verifier, the `proof-conversion` o1js verifier for these Groth16 proofs exposes all `inputCount` public values, which is five for SP1 proofs:

```
// proof-conversion: src/groth/recursion/zkp15.ts

async method(input: Field, PI: G1Affine, acc: G1Affine, full_pis: Array<FrC>)
    {
  const pi_hash = Poseidon.hashPacked(G1Affine, PI);
  const pis_hash = Poseidon.hashPacked(
    Provable.Array(FrC.provable, inputCount),
```

```
    full_pis
  );
  const acc_hash = Poseidon.hashPacked(G1Affine, acc);
  input.assertEquals(
    Poseidon.hashPacked(Provable.Array(Field, 3), [
      pi_hash,
      pis_hash,
      acc_hash,
    ])
  );
  // ...
  return { publicOutput: pis_hash };
},
```

Currently Groth16 proofs are not supported in the Mina smart contract, as the already mentioned on-chain check

```
const piDigest = Poseidon.hashPacked(Provable.Array(FrC.provable, 2), [
    pi0,
    pi1,
]);

piDigest.assertEquals(proof.publicOutput.rightOut);
```

enforces that the output of the circuit chain is a hash of two field elements rather than of five elements.

Impact

Three SP1 PLONK public inputs are not pinned to their intended legitimate values. The critical case is `pi3` (`vk_root`); `pi2` (`exit_code`) and `pi4` (`proof_nonce`) are discussed afterwards.

Verification key root `pi3 = vk_root`

The PLONK proof that SP1 produces is not a proof of a circuit that directly verifies correct execution of the underlying RISC-V program. Instead, the underlying shard proofs that constrain RISC-V execution get aggregated and compressed through a tree of recursive verifiers. The role of `vk_root` is to select which set of recursion verification keys is trusted by the in-circuit verifiers. Recursion programs that are intended to verify other recursion programs will pass upwards public values from any valid proof under a verification key included in `vk_root`. SP1 ships a constant `VK_ROOT_BYTES` (in `crates/verifier/src/lib.rs`) that is the Merkle root of the legitimate set. In earlier versions of SP1, `vk_root` was pinned to a constant by the wrap program, but in version 6.1.0 it is a PLONK public input and the legitimate-value check is the consumer's responsibility. The reference Solidity verifier (`crates/prover/assets/SP1VerifierPlonk.txt`) implements this responsibility against a hardcoded value:

```
// sp1/crates/prover/assets/SP1VerifierPlonk.txt
uint256 expectedVkRoot = uint256(VK_ROOT());
// ...
if (vkRoot != expectedVkRoot) {
    revert InvalidVkRoot();
}
```

With no constraints on `vk_root`, a malicious prover can utilize a root hash that includes the verification key of an attacker-authored recursion program that imposes no constraints on the program's public values. A top-level PLONK proof can then be constructed that attests to the existence of a valid proof for the attacker's recursion program, with the five public values exposed by the PLONK proof lifted from that underlying proof. This leaves the attacker with full control of the other four public values (all but `vk_root` itself).

In particular, the attacker can set `pi0` to the legitimate Nori `noriHeliosProgramPi0` constant and `pi1` to a `committed_values_digest` corresponding to any sequence of bytes they wish. This allows the attacker to settle fake attacker-chosen Ethereum state roots and deposit roots into the Mina bridge and to subsequently mint arbitrary amounts of bridged tokens to themselves.

Exit code `pi2 = exit_code`

The `exit_code` public value exposes the actual exit code of the underlying RISC-V guest program. The reference Solidity verifier rejects proofs where the exit code is not zero, indicating success:

```
// sp1/crates/prover/assets/SP1VerifierPlonk.txt
if (exitCode != 0) {
    revert InvalidExitCode();
}
```

The bridge not checking `pi2` means that valid proofs for executions where the underlying program panicked will be accepted. In the specific case of the audited version of `nori-bridge-head/nori-program`, the issue is latent:

```
// nori-bridge-head: nori-program/src/main.rs

pub fn main() {
    // Read zk input
    let encoded_inputs = sp1_zkvm::io::read_vec();

    // Decode inputs
    println!("Decoding inputs");
    let proof_inputs: ProofInputs<MainnetConsensusSpec>
    = serde_cbor::from_slice(&encoded_inputs).unwrap();
    println!("Decoded inputs");
```

```
    // Run nori sp1 helios zk program
    let proof_outputs = consensus_mpt_program(proof_inputs, true).unwrap();

    // Write zk output
    sp1_zkvm::io::commit_slice(&proof_outputs.to_bytes());
}
```

The `nori-program` writes values to the public values via exactly one call to `commit_slice`, at the end of `main`, with an unconditional write of fixed size (the type of `proof_outputs` is `ProofOutputs`, which is a struct of fixed size).

A panic before the call to `commit_slice` would thus leave the output digest corresponding to the empty bytestring. Finding a `(input_slot, input_store_hash, …, genesis_root)` byte sequence whose `parsePlonkPublicInputsProvable` image equals this fixed value would amount to a successful preimage attack on SHA-256 truncated to 253 bits, which is assumed to be infeasible.

Assuming soundness of `consensus_mpt_program`, reaching `commit_slice` without a panic implies that `proof_outputs` is valid. The only non-intended output that could occur due to a panic would then be if `commit_slice` itself were to panic. In this case, assuming no soundness bugs in the SP1 libraries, the output digest would either be as intended, or pose a similar unfeasible preimage attack.

The missing `exit_code == 0` check is therefore not exploitable against the present guest, but this depends on properties of the guest program.

Proof nonce `pi4 = proof_nonce`

The proof nonce `pi4` was introduced by SP1 in commit `4f75605c4` to allow creation of multiple distinct proofs even for identical programs and inputs. For soundness of the protocol, checks on `proof_nonce` are thus not necessary.

Groth16

Should the o1js Groth16 verifier in the `proof-conversion` repository be used in the Mina smart contract or anywhere else without adding pinning for the `pi2` and `pi3`, the same issue would occur.

Recommendations

The `pi3 = vk_root` public value must be enforced to be equal to the legitimate SP1 `VK_ROOT_BYTES` constant. While not currently an exploitable gap, we also strongly recommend checking that `pi2 = exit_code` is equal to zero.

One way to enforce both is to add corresponding constraints in the PLONK and Groth16 verifier circuits. An alternative would be to make this check the responsibility of the user, and thus to

expose these public values together with `pi0` and `pi1` in the PLONK circuits, and then enforce the checks in the on-chain bridge contract.

Remediation

This issue has been acknowledged by Nori, and a fix was implemented in commit 0352070b ↗.

The commit adds constraints to the chunked Groth16 and PLONK verifier circuits that pin `pi2` to zero and `pi3` to a constant.

3.3. [ec50d] Missing on-curve and subgroup checks in the Groth16 recursion verifier

Target	proof-conversion::groth::recursion		
Category	Soundness	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

The Groth16 verifier circuit (the sequence of recursion proofs `zkp0`, ..., `zkp15` under src/groth/recursion/) operates over two groups on the BN254 curve:

- G_1 is the full group $E(\mathbb{F}_p)$, which for BN254 has prime order r. On-curve membership is therefore equivalent to subgroup membership: a single on-curve check is sufficient.
- G_2 is the order-r subgroup of $E'(\mathbb{F}_{p^2})$ (the sextic twist of the original curve used by the BN254 pairing). The full group $E'(\mathbb{F}_{p^2})$ has cofactor much larger than 1, so on-curve membership does *not* imply order-r membership. Both an on-curve check and a subgroup-membership check are therefore required.

The verifier consumes three prover-controlled proof points: `negA` and `C` on G_1, and `B` on G_2.

The relevant types `G1Affine` and `G2Affine` are plain `Structs` over the field coordinates, neither with an overridden `check` function that could enforce the curve relation:

```
// proof-conversion: src/ec/index.ts
class G1Affine extends Struct({ x: FpA.provable, y: FpA.provable }) {}

// proof-conversion: src/ec/g2.ts
class G2Affine extends Struct({ x: Fp2, y: Fp2 }) {
  // ...
}
```

The only constraints applied to the parsed coordinates are the range-checks on the limbs of the underlying `FpA` elements that ensure they form a valid almost-reduced field element (`Fp.AlmostReduced`). The curve equation `y^2 = x^3 + 3` (for `negA`, `C`) and its `Fp2` twist analogue (for `B`) are never asserted, and no order-r subgroup-membership test is performed on `B`. The missing checks are therefore: on-curve for `negA` and `C`, and both on-curve and subgroup-membership for `B`.

These checks are also not enforced indirectly anywhere else in the verifier. The pre-pairing constraints `b_line.assert_is_tangent` / `b_line.assert_is_line` (src/lines/index.ts) enforce only line identities (for example $y - \texttt{lambda} \cdot x + \texttt{neg_mu} = 0$ and $2 \cdot \texttt{lambda} \cdot y = 3x^2$ for `assert_is_tangent`). As the line parameters `lambda` and `neg_mu` are prover controlled, these

constraints can be satisfied for any pair `(x, y)` by solving for `lambda` and `neg_mu` off-circuit, regardless of whether `(x, y)` lies on the curve. Inside the per-stage circuits, the proof points flow through `AffineCache` (src/lines/precompute.ts), which requires only that the point's `y` coordinate be nonzero (via `yp_prime * y = 1`). Across `zkp0`, ..., `zkp12`, every constraint on the proof points can be satisfied for any off-curve pair of canonical field elements. The only remaining assertion is the final pairing equality `f.assert_equals(Fp12.one())` at src/groth/recursion/zkp13.ts:50 — a check on the pairing relation, not on curve or subgroup membership of the inputs.

To confirm the absence of these checks end-to-end, we constructed a POC that (a) tampers `negA.y` to `(negA.y + 1) mod p` before the proof is parsed and (b) gates out the final pairing equality `f.assert_equals(Fp12.one())` (both the in-circuit assertion at `zkp13.ts:50` and the off-circuit sanity check at witness_tracker.ts:323), then runs the full recursion pipeline `zkp0`, ..., `zkp15` against the example fixtures at src/groth/example_jsons/. The full patch is:

```
-- a/src/groth/recursion/prove_zkps.ts
++ b/src/groth/recursion/prove_zkps.ts
@@ -28,7 +28,43 @@ import { VK } from '../vk_from_env.js';

 const args = process.argv;

const proof = parseProof(VK, args[3]);
// POC_OFFCURVE_NEGA: when set, rewrite the proof JSON to use an off-curve
    negA
// (y' = y + 1 mod p). This demonstrates that no on-curve check rejects the
// point through any of the pre-final checks. The 'final' pairing assertion
// (zkp13 + witness_tracker.zkp13) must be disabled separately for the demo.
const BN254_P =

    21888242871839275222246405745257275088696311157297823662689037894645226208583n;
let proofPathForParse = args[3];
if (process.env.POC_OFFCURVE_NEGA === '1') {
  const origJson = JSON.parse(fs.readFileSync(args[3], 'utf-8'));
  const xBig = BigInt(origJson.negA.x);
  const yBig = BigInt(origJson.negA.y);
  let yNewBig = (yBig + 1n) % BN254_P;
  if (yNewBig === 0n) yNewBig = (yBig + 2n) % BN254_P;
  const lhs = (yNewBig * yNewBig) % BN254_P;
  const rhs = (((xBig * xBig) % BN254_P) * xBig + 3n) % BN254_P;
  const onCurve = lhs === rhs;
  console.log('[POC_OFFCURVE_NEGA] original negA.y =', yBig.toString());
  console.log('[POC_OFFCURVE_NEGA] tampered negA.y =', yNewBig.toString());
```

```
  console.log(
    '[POC_OFFCURVE_NEGA] does tampered (x,y) satisfy y^2 = x^3 + 3 (mod p)?',
    onCurve
  );
  if (onCurve) {
    throw new Error(
      'POC sanity check: tampered point is still on the curve; pick a
          different offset'
    );
  }
  origJson.negA.y = yNewBig.toString();
  proofPathForParse = `${args[3]}.poc_offcurve.json`;
  fs.writeFileSync(proofPathForParse, JSON.stringify(origJson), 'utf-8');
  console.log(
    '[POC_OFFCURVE_NEGA] wrote tampered proof JSON to',
    proofPathForParse
  );
}

const proof = parseProof(VK, proofPathForParse);
 const auxWitness = AuXWitness.parse(args[4]);
 const workDir = args[5];
 const cacheDir = args[6];
-- a/src/groth/recursion/zkp13.ts
++ b/src/groth/recursion/zkp13.ts
@@ -47,7 +47,11 @@ const zkp13 = ZkProgram({
         );
         f = f.mul(shift);

        f.assert_equals(Fp12.one());
        // POC_OFFCURVE_NEGA: skip the final pairing assertion when the PoC
        // env flag is set, since a generic off-curve negA makes f != 1
        if (process.env.POC_OFFCURVE_NEGA !== '1') {
          f.assert_equals(Fp12.one());
        }

         acc.state.f = f;

-- a/src/groth/witness_tracker.ts
++ b/src/groth/witness_tracker.ts
@@ -320,7 +320,11 @@ class WitnessTracker {
```

```
        [Fp12.one(), w27, w27_sq]
      );
      f = f.mul(shift);
    f.assert_equals(Fp12.one());
    // POC_OFFCURVE_NEGA: skip the off-circuit pairing sanity check when the
    // PoC env flag is set, since a generic off-curve negA makes f != 1
    if (process.env.POC_OFFCURVE_NEGA !== '1') {
      f.assert_equals(Fp12.one());
    }

      this.acc.state.f = f;
      return this.acc.deepClone();
```

Running the pipeline with `POC_OFFCURVE_NEGA=1`, every stage produces a valid proof:

```
[POC_OFFCURVE_NEGA] does tampered (x,y) satisfy y^2 = x^3 + 3 (mod p)? false
valid zkp0?:  true
valid zkp1?:  true
valid zkp2?:  true
valid zkp3?:  true
valid zkp4?:  true
valid zkp5?:  true
valid zkp6?:  true
valid zkp7?:  true
valid zkp8?:  true
valid zkp9?:  true
valid zkp10?:  true
valid zkp11?:  true
valid zkp12?:  true
valid zkp13?:  true
valid zkp14?:  true
valid zkp15?:  true
```

The same POC structure straightforwardly extends to tampering `C` and `B`. For `B`, the `b_lines` (line coefficients) are computed off-circuit in `ProofClass.parse` via `computeLineCoeffs(B)` (src/lines/coeffs.ts); recomputing them from the tampered `B` keeps the line constraints `assert_is_tangent` / `assert_is_line` satisfied throughout the Miller loop, by construction.

Impact

The Groth16 verifier is the trust anchor for one of the two SNARK families consumable by the Nori bridge (the other being PLONK; see src/plonk/). The SP1 Groth16 path produces a final Mina proof whose `rightOut` field is consumed by `NoriTokenBridge.ethVerify` on the Mina side to authorize

state-root and deposit-root updates. If the Nori bridge is configured to use the Groth16 verifier, a break of this verifier therefore allows an attacker to forge arbitrary public values into the bridge's accepted state, with the same end-state consequences as documented in Finding 3.1. ↗.

The missing on-curve / subgroup checks invalidate the soundness argument the verifier depends on. All of the verifier's per-stage constraints — tangent/secant line identities, Frobenius identities, the residue-witness consistency relations — are formal polynomial relations that are satisfiable starting with any coordinate inputs, not just on-curve points. The single remaining soundness barrier is therefore the final pairing equality `f.assert_equals(Fp12.one())`. For this to be a sound Groth16 verification condition, the initial inputs have to be on-curve and, for `B`, in the appropriate subgroup.

Recommendations

Add constraints to the circuit that force the inputs to be on-curve and, for `B`, in the subgroup `G2`.

Concretely:

- **`negA` and `C` (in G_1):** add an in-circuit assertion that the point satisfies the BN254 curve equation. Since G_1 has prime order, an on-curve check alone is sufficient. The o1js `ForeignCurve` class already provides an `assertOnCurve()` method, so the simplest implementation is to either reroute `G1Affine` through the existing `bn254` curve type in `src/ec/g1.ts`, or to add an equivalent assertion directly on the `G1Affine` coordinates.
- **`B` (in G_2):** add both an on-curve check (the curve equation of the twisted curve, using the twist parameters already in src/towers/precomputed.ts) and a subgroup-membership check. The standard efficient construction is the Frobenius-based subgroup test of Dai et al., which decides membership using only the Frobenius endomorphism and a small number of point operations; the Frobenius primitive itself is already implemented as `G2Affine.frobenius()` and `G2Affine.negative_frobenius()` in src/ec/g2.ts.

Remediation

This issue has been acknowledged by Nori, and a fix was implemented in commit 67209dab ↗.

3.4. [d3034] Compromised admin is not prevented from minting or unlocking illegitimately

Target	NoriTokenBridge (Mina), NoriTokenBridge (Ethereum)		
Category	Protocol Risks	**Severity**	Critical
Likelihood	Low	**Impact**	High

Description

The Mina bridge is designed to hold up even against a compromised or malicious governance: a compromised admin should not be able to mint tokens that are not backed by real Ethereum deposits. The project describes the intended assumptions as follows:

> Using AdminKey (multisig), [the admin account] can update the values here: [...]. Worst case, [the admin] could maliciously set these to wrong values and "brick" the bridge. The Ethereum smart contract address is set at deployment and can't be modified, and since the Solidity contract is not upgradable, [they] can't create fake locks, so [..] shouldn't be able to mint tokens maliciously on the Mina side.

In the code, this intent is also stated, most explicitly in the doc comment on the immutable `genesisRoot` field, which singles out one way a compromised governance could otherwise mint unbacked tokens — redirecting the bridge to an attacker-controlled chain — and aims to prevent it:

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts

/**
 * Poseidon hash of the Ethereum chain genesis validators root.
 * Immutable after deploy — no admin setter exists by design.
 *
 * The Helios store hash (latestHeliusStoreInputHash*) is an opaque commitment
   over a
 * serialized light client store containing finalized headers, sync
   committees, and
 * optionally a next sync committee / best valid update. It must be upgradable
   because:
 *   1. The store serialization format may need to change as Helios evolves.
 *   2. A best-valid-update scenario (e.g. extended finality failure on
   Ethereum) would
 *      require a forced Helios store reconstruction and therefore a new store
   hash.
 *
```

```
 * That necessary upgradability is a governance attack surface: because the
    store hash is
 * opaque, a compromised governance update could swap it to one rooted on a
    different chain
 * with no visible on-chain evidence or verification. This field prevents that
    — `update`
 * verifies the proof's genesis root against this value, so governance can
    rotate the
 * store hash (subject to the strictly-increasing slot constraint) but cannot
    redirect
 * the bridge to a different chain. The genesis validators root is a
 * fixed, well-known constant that does not change across non-contentious hard
    forks.
 */
@state(Field) genesisRoot = State<Field>();
```

The claimed guarantee is that, although the store hash must stay admin-upgradable and is opaque, `update` checks the genesis validators root emitted by each proof against the immutable `genesisRoot`, so a rotated store hash cannot point the bridge at a different chain.

This protection does not work: the genesis root is not an anchor to the real chain. The bridge-head guest program performs an incremental store-to-store transition and never verifies that the store descends from genesis. The store, the genesis root, and the previous store hash are all witness inputs; the only check binding the store is its hash against the chained value:

```
// nori-bridge-head: nori-program/src/consensus.rs (consensus_mpt_program)

let ProofInputs {
    updates,
    finality_update,
    expected_current_slot,
    mut store,
    genesis_root,
    forks,
    store_hash: input_store_hash,
    contract_storage,
} = proof_inputs;

// ...

let calculated_prev_store_hash = sha256_hash_helios_store(&store)
    .map_err(|e| ProgramError::StoreHashingError(format!("Failed to hash
    store: {}", e)))?;
if calculated_prev_store_hash != input_store_hash {
    return Err(ProgramError::HashChainMismatch {
        expected: input_store_hash,
```

```
        actual: calculated_prev_store_hash,
    });
}

// ...

verify_update(update, expected_current_slot, &store, genesis_root, &forks)?;

// ...

verify_finality_update(&finality_update, expected_current_slot, &store,
    genesis_root, &forks)?;

// ...

let proof_outputs = ProofOutputs { /* ... */ genesis_root };
```

Both `verify_update` and `verify_finality_update` only pass `genesis_root` onwards to `verify_generic_update`, where the genesis root is used purely to derive the BLS signing domain.

The real root of trust is therefore the store-hash chain, seeded once at deploy. The genesis root only fixes the signing domain; it does not establish that the store's sync committees or headers belong to the real Ethereum chain. That trust would normally come from bootstrapping the store from a trusted checkpoint, which this program does not do — it accepts the store as an unauthenticated input.

Store-hash rotation.

A compromised admin can weaponize this directly, using only the power the comment considers safe. `updateStoreHash` sets the on-chain store hash to any value on an admin signature:

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts

@method async updateStoreHash(newStoreHash: Bytes32FieldPair) {
    await this.ensureAdminSignature();
    this.latestHeliusStoreInputHashHighByte.set(newStoreHash.highByteField);
    this.latestHeliusStoreInputHashLowerBytes.set(
        newStoreHash.lowerBytesField
    );
}
```

A compromised admin generates their own BLS sync committee, builds a `LightClientStore` containing it together with a fabricated finalized header carrying an attacker-chosen execution state root, and sets the on-chain store hash to that store's hash. They then submit an `update` whose finality update (and any sync-committee updates) are signed by the fabricated committee, using the genuine `genesis_root` for the domain. As the attacker controls the fabricated committee's keys, they can produce the required signatures, and can thus produce a proof for the

guest program with arbitrary deposit root contents. Using such a proof in an `update` transaction, the on-chain `genesisRoot` check will be passed, and `latestVerifiedContractDepositsRoot` updated to contain the fake deposits of their choosing. This is precisely the redirect to a fake chain that `genesisRoot` immutability was meant to prevent.

Program and recursion verification key swap.

The genesis-root failure is not even required for a takeover. Separately from it, the admin holds powers that were presumably not meant to allow unbacked minting but do. One is that the identity of the accepted guest program is itself admin-mutable, giving a more direct route:

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts

@method async updateNoriHeliosProgramPi0(newPi0: Fr0) {
    await this.ensureAdminSignature();
    this.noriHeliosProgramPi0.set(newPi0);
}

@method async updateProofConversionPi2(newPi2: Field) {
    await this.ensureAdminSignature();
    this.proofConversionPi2.set(newPi2);
}
```

In `ethVerify` the converted proof is verified against a fixed verification key, which is intended to be for the `node` compressor circuit in `src/compressor/compressor.ts` in `proof-conversion`. However, this circuit bakes in neither the verification keys it recursively verifies proofs against, nor which guest program the SP1 proof is ultimately verified against. Only commitments are exposed, which need to be pinned on-chain. This is done against `noriHeliosProgramPi0` for pinning the guest program and against `proofConversionPi2` for pinning the recursion verification keys.

An admin can change both commitments. Changing either lets the on-chain checks accept a constraint-less recursion circuit or guest program, allowing a compromised admin to produce accepted `update`s carrying deposit roots with arbitrary contents.

By any of these routes a compromised admin can produce an `update` carrying a deposit root with contents of their choosing, then use `noriMint` to mint an arbitrary amount, for the invented deposits, to an address they control. Those tokens could then be withdrawn on the Ethereum side using the usual mechanism, draining the bridge.

This list is not exhaustive: a compromised admin may have further avenues to mint themselves illegitimate tokens, for instance by changing the `FungibleToken` verification key.

Ethereum side.

The same structural pattern exists on the Ethereum bridge. The function `unlockTokens` trusts two contracts for verification, `stateSettlement` and `accountValidation`, and `setAlignedContracts` lets the operator repoint the bridge at replacements, which could include contracts that approve any input, which would then allow the compromised admin to drain the locked pool.

Impact

The intention that a compromised admin should not be able to drain the bridge on the Ethereum side, or mint illegitimately on the Mina side, but at worst can only brick the bridge, is not achieved by the current design. The admin account/contract has several independent avenues to drain the bridge.

Recommendations

Document for the project and for users what trust assumptions are made regarding the admin. To achieve the intended threat model, in which the admin account/contract should not be able to mint/unlock illegitimately, design changes are necessary.

Remediation

This issue has been acknowledged by Nori. Commit 0a011766 ↗ in the nori-bridge-sdk repository and commit 573dfa9f ↗ in the nori-bridge-head repository record in the changelog the decision to address this finding by documenting the trust assumptions made regarding the admin, while retaining the in-principle on-chain ability of the admin to mint/unlock illegitimately. Furthermore, the commits remove the `genesisRoot` state variable as well as the associated comment from the Mina smart contract, as this logic did not achieve the intended aim to restrict admin powers. The changelog states:

> What constrains the role is procedural rather than cryptographic. The admin is FROST-based multisig, so a change to any of these values requires commitment from a quorum of the signers. The project has a governance process for approving upgrades, and the multisig is held by a set of parties that are expected to be independent and to follow that process. The trust assumption is that the multisig signers will not collude to approve an upgrade that would allow them to mint unbacked tokens or unlock illegitimately.
>
> [...]
>
> As part of a solution to this finding, a standalone document is being prepared to record the trust assumptions that govern the admin role, and to describe the procedures that are expected to be followed to maintain the bridge's integrity. This document will serve as a reference for users and developers, clarifying the limits of what a compromised admin can do and the safeguards in place to prevent abuse.

Key custody and admin procedures were out of scope for this engagement and not reviewed by Zellic.

3.5. [4279a] Unvalidated `oldestAction` in `dispatchAndEvict` can permanently brick ETH to Mina minting

Target	bridge-sdk::mina::NoriTokenBridge		
Category	Protocol Risks	**Severity**	High
Likelihood	High	**Impact**	High

Description

The `NoriTokenBridge` Mina smart contract maintains a rolling window of up to `maxWindow` (32) verified Ethereum deposit Merkle roots using an o1js `Reducer`. Each successful `update` transaction calls `dispatchAndEvict`, which dispatches `input.verifiedContractDepositsRoot` as a new action and slides `windowStart` forward by one step if the window is already full, so that the window still covers the last 32 roots.

The docstring and comments of `dispatchAndEvict` state that, when the window is full, the caller supplies the oldest dispatched action within the window as a witness and the contract verifies the hash chain before advancing:

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts

/**
 * ...
 *
 * When the window is full, the caller must provide the oldest action as a
 * witness. The contract verifies the hash chain:
 *   advanceActionState(windowStart, singleActionInnerHash(oldestAction))
 * and advances windowStart by one step, keeping the window size constant.
 */
private dispatchAndEvict(depositRoot: Field, oldestAction: Field) {
    // Dispatch the new deposit root as an action
    this.reducer.dispatch(depositRoot);

    let windowStart = this.windowStart.getAndRequireEquals();
    let windowSize = this.windowSize.getAndRequireEquals();

    const isFull = windowSize.greaterThanOrEqual(maxWindow);

    // Compute the advanced windowStart by verifying the oldest action chains
    correctly.
    // If isFull is false, this computation is ignored (oldestAction can be
    anything).
```

```
    const innerHash = singleActionInnerHash(oldestAction);
    const advancedStart = advanceActionState(windowStart, innerHash);

    // Conditionally advance: if full, slide the window; otherwise keep start.
    this.windowStart.set(Provable.if(isFull, advancedStart, windowStart));
    // If full: evict 1 + add 1 = same size. If not full: size + 1.
    this.windowSize.set(Provable.if(isFull, windowSize, windowSize.add(1)));
}
```

In the implementation, `oldestAction` is provided by the caller to `update`, where it is not used beyond passing it unverified to `dispatchAndEvict`. Here, it is only used in the evaluation of the new `windowStart` value using `singleActionInnerHash` and `advanceActionState`. Neither of which carries out any verification that `oldestAction` was indeed the true next action in the reducer history after the existing `windowStart`, instead merely mechanically computing an iterated hash that depends on `oldestAction`:

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts

function singleActionInnerHash(action: Field): Field {
    const eventHash = hashWithPrefix('MinaZkappEvent******', [action]);
    return hashWithPrefix('MinaZkappSeqEvents**', [emptyActionsHash,
    eventHash]);
}

function advanceActionState(state: Field, innerHash: Field): Field {
    return hashWithPrefix('MinaZkappSeqEvents**', [state, innerHash]);
}
```

Impact

The `windowStart` state variable is used in the `noriMint` method to load actions from the reducer starting from the `windowStart` action state, and then reduce over them:

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts

const windowStart = this.windowStart.getAndRequireEquals();
const actions = this.reducer.getActions({ fromActionState: windowStart });

const depositInWindow: Bool = this.reducer.reduce(
    actions,
    Bool,
    (found: Bool, action: Field)
    => found.or(action.equals(contractDepositSlotRoot)),
    Bool(false),
    { maxUpdatesWithActions: maxWindow }
```

```
);
```

The `getActions` reducer function returns a `MerkleList` whose empty list hash, used in particular when iterating, will be the passed circuit variable `windowStart`. The list entries returned by `getActions` instead are not constrained and are merely assigned by the prover, so in principle the prover could witness action list entries that are unrelated to `windowStart`.

However, `reduce` iterates from the given empty hash (so `windowStart`), constraining a hash chain ending in `actionLists.hash`, with a transaction precondition being emitted that binds this circuit variable to the contract's on-chain `actionStates`:

```
// o1js: src/lib/mina/v1/actions/reducer.ts

reduce<S>(
  actionLists: MerkleList<MerkleList<A>>,
  stateType: Provable<S>,
  reduce: (state: S, action: A) => S,
  state: S,
  {
    maxUpdatesWithActions = 32,
    maxActionsPerUpdate = 1,
    skipActionStatePrecondition = false,
  } = {}
): S {
  // ...
  if (!skipActionStatePrecondition) {
    // the actionList.hash is the hash of all actions in that list, appended
    to the previous hash (the previous list of historical actions)
    // this must equal one of the action states as preconditions to build a
    chain to that we only use actions that were dispatched between the current
    on chain action state and the initialActionState
    contract.account.actionState.requireEquals(actionLists.hash);
  }

  const listIter = actionLists.startIterating();

  for (let i = 0; i < maxUpdatesWithActions; i++) {
    // ...
  }

  // important: we check that by iterating, we actually reached the claimed
    final action state
  listIter.assertAtEnd();

  return state;
},
```

An attacker can thus permanently brick ETH to Mina minting by calling `update` with a bogus `oldestAction` when the window is full:

1. Attacker calls `update` with a valid SP1 transition proof (permissionless for holders of a valid proof, which in turn is computable by an attacker) and a bogus `oldestAction`.
2. The `windowStart` state variable becomes `H('MinaZkappSeqEvents**', [windowStart, H('MinaZkappSeqEvents**', [emptyActionsHash, H('MinaZkappEvent******', oldestAction)])])`, that is, a point on a corrupted branch chain of the action-state hash chain, not the branch the reducer actually appended deposit roots to.
3. Subsequent `update` transactions continue to record deposit-root actions on the canonical chain, while advancing `windowStart` on the branch chain. The assumed infeasibility of computing the structured Poseidon hash collision that would be required implies that it will not be possible for anyone to exhibit the tip of the branch chain as an ancestor of any state occurring in the canonical chain.
4. All `noriMint` calls thus fail the `contract.account.actionState.requireEquals(actionLists.hash);` precondition for any internally consistent witnessed action list.

Recommendations

We recommend verifying in `dispatchAndEvict` that `oldestAction` is the actual oldest action after `windowStart` in the action chain, by `reduce`ing from `windowStart` and extracting the oldest action being iterated over.

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit 39b7d0a9 ↗ in the nori-bridge-sdk repository. Zellic did not review this change (see section 6.1. ↗).

3.6. [a3850] The committed deposit root is not constrained to be complete

Target	bridge-head::nori-program::mpt		
Category	Protocol Risks	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

To bridge from Ethereum to Mina, a user locks funds on the Ethereum `NoriTokenBridge`, which records the deposit in the cumulative mapping `lockedTokens[codeChallenge]`. On the Mina side, `noriMint` lets that user mint, gated on the user's deposit being a member of a rolling window of up to `maxWindow` (32) verified deposit roots. Each root is committed to Mina by an `update` call carrying an SP1 proof whose output `verified_contract_storage_slots_root` (surfaced on Mina as `verifiedContractDepositsRoot`) is dispatched into that window.

The issue discussed in this finding stems centrally from the fact that this deposit root need not include all deposits. Instead, it is a freshly built Poseidon Merkle tree over whatever storage slots the prover supplies, and the SP1 guest verifies only the inclusion of those slots; it never enforces that the tree is complete.

The root is produced by `verify_storage_slot_proofs`. After verifying the contract account against the execution state root, it builds the tree purely from the supplied `contract_storage.storage_slots`, returning the all-zero hash when the prover supplies none (see 3.8. ↗ for why zero is used as the root hash of an all-padding tree):

```
// nori-bridge-head: nori-program/src/mpt.rs

// Optimisation, skip doing the MPT proof if we have no storage slots in this
    window
let n_leaves = contract_storage.storage_slots.len();
if n_leaves == 0 {
    return Ok(FixedBytes::default())
}

// ...

let mut merkle_nodes = Vec::with_capacity(padded_size);

for slot in contract_storage.storage_slots {
    // ...
    proof::verify_proof(
        contract_storage.expected_value.storage_root,
```

```
        key_nibbles,
        Some(rlp_encoded_value),
        &slot.mpt_proof,
    )
    .map_err(|e| MptError::InvalidStorageSlotProof {
        slot_key: key,
        reason: e.to_string(),
    })?;
    // ...
    merkle_nodes.push(slot_merkle_leaf);
}

// Calculate the root hash
let root = fold_merkle_left(&mut merkle_nodes, padded_size, depth,
    &get_merkle_zeros());
```

Each supplied slot is checked to exist under the contract's storage root, but nothing constrains what is omitted, so a proof carrying zero deposit slots, and a Merkle root hash indicating an empty tree (only padding), is valid.

As `update` is permissionless, any party, not only the operator, can advance the bridge head with a valid proof carrying an empty or pruned deposit root.

Impact

Active preemption of the mint path. Each `update` must advance the Ethereum slot (`latestHead`):

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts

const currentSlot = this.latestHead.getAndRequireEquals();

// ...

// Verification of slot progress.
proofHead.assertGreaterThan(
    currentSlot,
    'Proof head must be greater than current head.'
);

// ...

// Update contract values
this.latestHead.set(proofHead);
```

Thus every `update` carries a precondition binding it to the previous value of the state variable `latestHead`, which the update then changes. Hence, if an attacker front-runs the legitimate

operator's update with a successful `update` of its own, the operator's transaction will be rejected. Note that the attacker can begin working on a proof that will update `latestHead` from N to $N+1$ before the `update` that advances the on-chain state of `latestHead` to N got finalized on Mina. Indeed, if they anticipate that the next `update` will advance `latestHead` to N and slot $N+1$ has been finalized on Ethereum, they can already begin computing the proof. It thus appears plausible that a determined attacker can preempt legitimate updates and persist in landing `update` calls with empty deposit root on-chain, preventing the legitimate operator or other parties from placing non-empty deposit roots.

Under such a sustained attack, the mint path is denied to all users for as long as the attack continues. The funds are not permanently burned, however. The Ethereum contract's `lockedTokens[codeChallenge]` is cumulative and never deleted: the deposit still exists in Ethereum storage at any later output block, and the nori-program SP1 guest program checks each slot against the output block's storage root rather than the block at which it was locked. Because of this, once the attacker stops, an unopposed self-rescue or the operator can drain the backlog. Thus the impact is an attacker-controlled outage of the mint path rather than irreversible loss.

Passive omission. Even without an attacker there is no guarantee that any given deposit is ever committed. A selectively censoring, buggy, or offline operator can omit a deposit, so that it is never mintable via updates submitted by the operator. The user's only recourse is a self-rescue run in which they race the operator with an `update` of their own that includes their deposit, a high bar for an ordinary user, though possible when unopposed.

Data-availability dependency. To mint against an update, a user needs the exact leaf set and order of that update's tree. Data availability is thus a point to consider, though out of scope for this audit.

Recommendations

The root cause is that the committed deposit root is not required to encompass all intended deposits as reflected by the Ethereum state. The fix is to enforce such a requirement in the SP1 guest program, so that any valid `update` must carry the deposits that Ethereum state records.

How to adjust the design of the guest program and potentially also the Ethereum contract is a significant design question. There is however one concrete footgun to be aware of. Completeness must be enforced incrementally rather than all-at-once. A rule of "include everything since the last update" is itself a denial-of-service vector: by spamming cheap locks, an attacker can either exceed the Merkle tree's 2^{16}-leaf limit for a single update, or produce deposits faster than the operator can prove them.

Remediation

This issue has been acknowledged by Nori. Commit 0a011766 ↗ in the nori-bridge-sdk repository and commit 573dfa9f ↗ in the nori-bridge-head repository modify the design of the Ethereum to Mina direction of the bridge substantially to fix the root cause of this finding.

With these changes, a new Ethereum smart contract is introduced, `NoriProofRequestQueue`, which holds an append-only queue of requests, with each request consisting of the address of the

requesting account, the slot in that account's storage to read, as well as up to two words of attached data, called collection keys. The `lockTokens` function of `NoriTokenBridge` now calls into `NoriProofRequestQueue` to queue such a request for the slot `lockedTokens[codeChallenge]` to which tokens are being added, and with `codeChallenge` as the attached collection key. As each request records the caller as the `target`, and the only callpath through which `NoriTokenBridge` calls the request queue is via the described call in `lockTokens`, the only requests in the `NoriProofRequestQueue` queue whose target is the `NoriTokenBridge` contract can be requests arising from a `lockTokens` call, and the slot to read and collection key(s) must be the `lockedTokens[codeChallenge]` slot and `codeChallenge` from exactly that `lockTokens` call.

The nori-program SP1 guest program is updated to expose in its public values `input_queue_cursor` and `output_queue_cursor`. These are used as indices into the append-only queue of the `NoriProofRequestQueue` contract. Previously, the program constructed a Merkle tree of deposits by reading values of the `NoriTokenBridge` contract's `lockedTokens` mapping directly, with the prover being able to choose which keys to include. This logic is replaced by instead building a Merkle tree of requests from the `NoriProofRequestQueue` contract. Instead of allowing the prover to choose which requests to include, they must include precisely the requests starting from index `input_queue_cursor` and up to `output_queue_cursor - 1`. Additionally, `output_queue_cursor` is enforced to either be the current (as per the Ethereum checkpoint updated to) on-chain `head` pointer of the queue contract, meaning that the update includes all outstanding requests since `input_queue_cursor`, or to be `input_queue_cursor + MAX_BATCH`, where `MAX_BATCH` is a fixed constant. This enforces every update to make progress on queued requests, if the Ethereum block being updated to contains outstanding requests. For each request, the indicated slot is read from the indicated target's storage, and a Merkle tree is built including all requests in the batch, with that read value replacing the slot key of the raw request from the queue. The program does not perform any validity checks on the request's data, for example checking that the arithmetic values are bounded, or that the target address is the one of the `NoriTokenBridge`, and also supports targets whose account does not exist (anymore) or slots that are not present in the relevant storage trie. This is in fact an important property, as including some of these checks could open a possible denial of service attack vector where an attacker submits requests to the queue from an account other than `NoriTokenBridge` that deliberately will fail the validation.

The `NoriTokenBridge` Mina smart contract was changed to keep track of a new state variable `queueCursor`, which starts at zero, with every update checking that the public value `input_queue_cursor` matches the existing `queueCursor`, and then after applying the update changing `queueCursor` to `output_queue_cursor`. This ensures that at each point in time, all requests in the queue before `queueCursor` have appeared previously in a Merkle tree dispatched by `update` on the Mina contract. Finally, as the Merkle tree published by nori-program now includes all requests, including those not from the `NoriTokenBridge` Ethereum smart contract, the `noriMint` method is changed to check that `target` of the entry that is to be used is equal to the address `ethTokenBridgeAddress`.

In our review of this remediation we found this new design to be sound, and agree that it resolves the fundamental root cause discussed in this finding. We provided Nori with additional feedback regarding the implementation as well as further considerations on the design. This includes in particular a significant vulnerability stemming from bugs in the alloy-trie dependency (along with further robustness concerns in the alloy-trie and related alloy-rlp dependencies) as well as feedback on selection of the value `MAX_BATCH`. Additionally, we provided a discussion of potentially

remaining avenues of an attacker to slow down the bridge by making each update advance only the minimum permitted, a single checkpoint. The changes discussed here removes the ability of a frontrunning attacker to place updates without including requests outstanding as of the Ethereum block that the update advances to, but doesn't constrain how recent that block must be. Slowing the bridge by only advancing a single checkpoint affords the attacker significantly less than the attack the original issue allowed. We provided Nori with suggestions for how to quantify the remaining risk and how to potentially adjust settings to change the tradeoff.

3.7. [1eb72] Missing in-circuit head-advancement and checkpoint-slot assertions

Target	bridge-head::nori-program::consensus		
Category	Business Logic	**Severity**	Medium
Likelihood	Low	**Impact**	Medium

Description

The upstream `sp1-helios` guest program asserts, in-circuit, that each transition advances the finalized head and lands on a checkpoint slot, with the following checks that appear after updates have been applied to `store`:

```
// From sp1-helios: light_client.rs
assert!(
    store.finalized_header.beacon().slot > prev_head,
    "New head is not greater than previous head."
);
assert!(
    store.finalized_header.beacon().slot % 32 == 0,
    "New head is not a checkpoint slot."
);
```

The Nori guest (`consensus_program` / `consensus_mpt_program` in `nori-program/src/consensus.rs`) does **not** have these assertions. This is acknowledged in the comment:

```
// nori-bridge-head: nori-program/src/consensus.rs
// Should do an assertion here to ensure we have increased our head (we do
    check this downstream later)
```

The checks were instead relocated out of the proof and into off-chain native validation and the on-chain Mina consumer:

- Off-chain bridge-head validation (`nori/src/rpcs/consensus/mod.rs`, `prepare_consensus_mpt_proof_inputs` with `validate = true`) enforces strict slot increase, checkpoint, nonzero next committee, and bootstrap:

```
// nori-bridge-head: nori/src/rpcs/consensus/mod.rs
if validate && output_slot <= input_slot {
    return Err(anyhow::anyhow!(
        "Output slot {} was not greater than input slot {}",
```

```
        output_slot,
        input_slot
    ));
}

// ...
// Block non-checkpoint slots (they prevent bootstrapping on restart)
// ...
if validate && output_slot % 32 > 0 {
    return Err(anyhow::anyhow!(
        "Output slot {} was a non-checkpoint slot. Preventing this as it
    prevents bootstrapping if we go offline.",
        output_slot,
    ));
}

if validate && proof_outputs.next_sync_committee_hash == B256::ZERO {
    return Err(anyhow::anyhow!(
        "Next sync committee was zero. Preventing this as it could stop the
    recovery of the input_store_hash when restarting.",
    ));
}

// Lines Skipped...

// Block non-checkpoint slots (ones where we fail to actually bootstrap by
    trying it explicitly)
// This re-enforces the output_slot % 32 > 0 validation check.
// I addition to checking the output_slot number % 32 lets try to bootstrap
    from this slot explicitly
if validate {
    Client::<S, R>::bootstrap_from_slot(&url, output_slot).await
    .map_err(|e| anyhow::anyhow!(
        "Failed to bootstrap from slot {} using {}. Preventing the use of this
    output_slot as it could lead to a stall of the bridge:\n{}",
        output_slot, url, e
    ))?;
}
```

- On-chain Mina validation (`nori-bridge-sdk/contracts/mina/src/NoriTokenBridge.ts`, `update`) enforces slot increase relative to the on-chain previous slot, and nonzero next committee:

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts
// Verification of slot progress.
proofHead.assertGreaterThan(
```

```
    currentSlot,
    'Proof head must be greater than current head.'
);

// Verification that next sync commitee is non zero (could brick the bridge
    head otherwise)
let nextSyncCommitteeZeroAcc = new Field(0);
for (let i = 0; i < 32; i++) {
    nextSyncCommitteeZeroAcc = nextSyncCommitteeZeroAcc.add(
        input.nextSyncCommitteeHash.bytes[i].value
    );
}
nextSyncCommitteeZeroAcc.assertNotEquals(new Field(0));
```

Impact

The on-chain checks enforce slot increase and a nonzero next committee, so there is no impact to the overall protocol of moving these checks from the guest program to the Mina contract.

The check that the finalized header slot is divisible by 32 does not occur in the Mina contract. For honest provers who prepare proofs via code going through the `prepare_consensus_mpt_proof_inputs` function mentioned above, with `validate` set to `true`, the out-of-circuit off-chain checks prevent accidental usage of such finalized headers.

However, malicious provers need not honor these checks. Because `update` transactions can be submitted by anyone, a malicious prover can place an update on-chain where the new finalized header slot is not divisible by 32.

Three lines of evidence indicate a non-checkpoint head is problematic for the operator: the comments in the code; the analogous in-circuit assertion in sp1-helios; and the developers' confirmation to us that such a head produced "LC bootstrap unavailable" RPC errors that bricked the bridge after a process crash.

We did not attempt to reproduce this issue ourselves. Nor did we investigate whether, should an attacker submit an `update` with a not divisible by 32 finalized header slot, the operator's process would then require the kind of bootstrap procedure in which the problem occurs. Updates submitted by anyone other than the honest operator are not the intended flow — let alone updates whose finalized header slot is not a checkpoint slot. It therefore appears plausible that the operator might reach a state where a restart would be required and the issue would surface. If this is the case, then if an attacker can place such an update on-chain, this would halt the bridge, preventing the legitimate operator from posting further updates via their normal flow.

It does happen regularly in practice that proposers miss checkpoint slots, which then go empty. In such a case, an update will be constructable with `store.finalized_header.beacon().slot` (after having applied updates to `store`) not being divisible by 32. As the legitimate operator will skip this epoch due to their checks, an attacker waiting for such an event will have little trouble in producing a proof for this epoch faster than the legitimate operator will produce a proof for the next epoch.

Epochs where the checkpoint slot was missed do not appear to be very rare, for example at the time of writing, the last such case occurred only about an hour prior ↗.

The immediate result of such an attack will be that the operator will be unable to continue posting legitimate updates on-chain. However, as the admin has the power to update the store hash on-chain, it is possible for the admin to un-brick the bridge:

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts

// we need it in case Helios changes it's store structure
@method async updateStoreHash(newStoreHash: Bytes32FieldPair) {
    await this.ensureAdminSignature();
    this.latestHeliusStoreInputHashHighByte.set(newStoreHash.highByteField);
    this.latestHeliusStoreInputHashLowerBytes.set(
        newStoreHash.lowerBytesField
    );
}
```

Note though that the documentation comment for this function indicates its use is intended for other situations in which a store hash update would be necessary. See also 3.4. ↗.

After the admin updates the store hash, the attacker could keep repeating this temporary denial of service with new epochs where the proposer missed the checkpoint slot.

While the admin can un-brick the bridge with a manual transaction after an attack, such attacks would amount to disruptive temporary denial-of-service attacks, which could lead to economic loss.

Recommendations

Reintroduce the assertion `output_slot % 32 == 0` in the guest program.

Also consider reintroducing the `output_slot > input_slot` assertion, so the proof is self-contained and matches the upstream `sp1-helios` security statement, as well as adding the `next_sync_committee_hash` assertion, rather than depending on the specific on-chain consumer. If the checks are intentionally kept off-circuit for flexibility, document the invariant that any proof consumer must re-enforce these assertions explicitly.

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit 29280373 ↗ in the nori-bridge-head repository. Zellic did not review this change (see section 6.1. ↗).

3.8. [a2090] Merkle zero index reversed

Target	bridge-sdk::o1js-zk-utils::merkleTree, hash::merkle_poseidon_fixed		bridge-head::nori-
Category	Integration Risks	**Severity**	Medium
Likelihood	Low	**Impact**	Low

Description

The `getMerkleZeros` function is a helper that caches the Merkle roots for binary Merkle trees of all zero leaves, to avoid redundant computation. Specifically, it returns an array `zeros` where `zeros[i]` is the Poseidon root of an all-zero subtree of depth `i` (`zeros[0] = 0`, then `zeros[i] = H(zeros[i-1], zeros[i-1])`). The array runs from the smallest dummy subtree to the largest:

```
// nori-bridge-sdk: o1js-zk-utils/src/merkle-attestor/merkleTree.ts

export function getMerkleZeros(depth: number): Field[] {
    const zeros: Field[] = [];

    // Start with zeros[0] = Field(0)
    zeros.push(Field(0));

    for (let i = 1; i < depth + 1; i++) {
        // Each next zero is hash of the previous zero with itself
        zeros.push(Poseidon.hash([zeros[i - 1], zeros[i - 1]]));
    }

    return zeros;
}
```

The relevant Merkle trees are perfect binary trees. If there are fewer entries to store than the total number of leaves, the remaining leaves are padded with zero. As an optimization, when building the Merkle tree from the leaves, root nodes of subtrees with all-zero leaves are not computed but assigned from the cached precomputed values returned by `getMerkleZeros`. A standard Merkle fold would take that value as `zeros[depth - 1]`, where `l` is the level of the node. Nori's builders instead assign root nodes of dummy subtrees at level `level - 1` with `zeros[level]`, which is a different way to apply the same cache and yields a different root than conventional padded-tree Merkle code:

```
// nori-bridge-sdk: o1js-zk-utils/src/merkle-attestor/merkleTree.ts
```

```
export function buildMerkleTree(
    merkleLeaves: Field[],
    paddedSize: number,
    depth: number,
    zeros: Field[]
): Field[][] {
    const nLeaves = merkleLeaves.length;
    const missing = paddedSize - nLeaves;

    // Clone and pad leaves
    const leaves = merkleLeaves.slice();
    for (let i = 0; i < missing; i++) {
        leaves.push(Field(0));
    }

    const merkleTree: Field[][] = new Array(depth + 1);
    merkleTree[depth] = leaves;

    let nNonDummyNodes = nLeaves;

    // Build tree from leaves upward
    for (let level = depth; level > 0; level--) {
        const childLevel = merkleTree[level];
        const parentWidth = 1 << (level - 1);
        const parentLevel: Field[] = new Array(parentWidth);

        for (let i = 0; i < parentWidth; i++) {
            const leftIdx = 2 * i;

            if (leftIdx >= nNonDummyNodes) {
                // Both left and right dummy nodes, use zeros cache
                parentLevel[i] = zeros[level];
            } else {
                const rightIdx = leftIdx + 1;
                // Atleast one of left and right are real.
                parentLevel[i] = Poseidon.hash([
                    childLevel[leftIdx],
                    childLevel[rightIdx],
                ]);
            }
        }

        merkleTree[level - 1] = parentLevel;

        // Shrink count of non-dummy nodes for next iteration
        nNonDummyNodes = Math.floor((nNonDummyNodes + 1) / 2);
```

```
    }

    return merkleTree;
}
```

Note that the implementation computes all nodes of the Merkle tree, starting from the leaf level, with leaves padded with zero. The code thus shows the clear intention of computing the root of a perfect binary Merkle tree with leaves (all at maximal depth, as the tree is perfect) padded with zero, rather than of a full but potentially non-perfect binary Merkle tree with dummy leaves inserted at non-maximal depths for padding. This makes the usage of the cached values incoherent rather than merely an unusual choice for dummy leaf values.

The same pattern appears in `nori-hash/src/merkle_poseidon_fixed.rs` of the nori-bridge-head repository (`zeros[level]` in `fold_merkle_left`, `build_merkle_tree`, and `get_merkle_path_from_leaves`, with `zeros` computed in `calculate_zeros`, then stored in a file accessed via `get_merkle_zeros`).

Impact

The functions `buildMerkleTree`, `foldMerkleLeft`, and `getMerklePathFromLeaves`, as well as their Rust analogues, implement a nonstandard Merkle fold over padded trees: they use the all-zero cache in a way that does not match standard Merkle root computation, so their outputs differ from a conventional implementation whenever the tree contains an all-dummy subtree of positive height unequal to $\frac{\texttt{depth}-1}{2}$. Within Nori this is internally consistent, since bridge-head (`merkle_poseidon_fixed.rs`) and witness builders using o1js-zk-utils (`merkleTree.ts`) share the same fold, so at the time of audit, there is no immediate completeness issue.

The risk lies in integration and maintenance: operators or future code that builds deposit witnesses with a conventional Merkle implementation will compute different roots and paths than the bridge-head proof commits to `verifiedContractDepositsRoot`, causing `noriMint` to fail.

Recommendations

We recommend coordinating a fix across both `o1js-zk-utils`, `nori-hash`, and other places in which the nonstandard convention is used, to ensure the results match the standard Merkle root computation. Note that applying the fix on only one side of the pipeline (TypeScript or Rust) would introduce inconsistency across the existing pipeline, and lead to completeness issues.

Usage of this convention is not limited to the functions mentioned above, for example `verify_storage_slot_proofs` contains an optimization that uses Nori's nonstandard convention as well, using zero as the root hash of a Merkle tree that consists entirely of padding:

```
// nori-bridge-head: nori-program/src/mpt.rs

// Optimisation, skip doing the MPT proof if we have no storage slots in this
    window
```

```
let n_leaves = contract_storage.storage_slots.len();
if n_leaves == 0 {
    return Ok(FixedBytes::default())
}
```

This example is not exhaustive, there may be more occurrences of such optimizations, and so we recommend a careful review of all code that involves Merkle tree computations to ensure all locations are modified in a compatible manner.

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit f8270774 ↗ in the nori-bridge-sdk repository. Zellic did not review this change (see section 6.1. ↗).

3.9. [41428] In-flight mints are invalidated by the next `update`

Target	bridge-sdk::mina::NoriTokenBridge		
Category	Protocol Risks	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

The `NoriTokenBridge` Mina smart contract maintains a rolling window of up to `maxWindow` (configured to be 32) verified Ethereum deposit roots using an o1js `Reducer`. Each `update` dispatches the latest `verifiedContractDepositsRoot` as a new action and, once the window is full, evicts the oldest root by advancing `windowStart`. The intent of this window is to give users a grace period of several hours during which they can mint against a deposit, rather than having to mint within the narrow interval between two consecutive updates.

To mint, `noriMint` checks that the user's deposit root is a member of the current window. It does so by reading `windowStart` and folding the action log from there with the reducer:

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts

const windowStart = this.windowStart.getAndRequireEquals();
const actions = this.reducer.getActions({ fromActionState: windowStart });

const depositInWindow: Bool = this.reducer.reduce(
    actions,
    Bool,
    (found: Bool, action: Field)
    => found.or(action.equals(contractDepositSlotRoot)),
    Bool(false),
    { maxUpdatesWithActions: maxWindow }
);
```

This snippet binds the resulting transaction to on-chain state in two independent ways:

1. The call `this.windowStart.getAndRequireEquals()` adds a strict-equality precondition on the `windowStart` app-state field. This means that at apply time the on-chain value must equal the value witnessed at proof-build time exactly.
2. The `reducer.reduce` call (with `skipActionStatePrecondition` left at its default) adds, internally, a precondition on the account's action state:

```
// o1js: src/lib/mina/v1/actions/reducer.ts

if (!skipActionStatePrecondition) {
  contract.account.actionState.requireEquals(actionLists.hash);
}
```

Unlike app state, the action state is checked against a five-slot history rather than the single latest value: the Mina protocol stores the last five action-state hashes and the precondition is satisfied if the pinned value matches any of them. The head advances on every action-emitting account update, but the history only rotates when a dispatch lands in a new slot. A mint's pinned head therefore survives roughly four later-block updates before it falls out of the window.

The action-state precondition (2) thus has a tolerance of about four updates. The `windowStart` precondition (1), however, has zero tolerance. The reason is that `dispatchAndEvict` advances `windowStart` on every update once the window is full:

```
// nori-bridge-sdk: contracts/mina/src/NoriTokenBridge.ts

const isFull = windowSize.greaterThanOrEqual(maxWindow);
// ...
this.windowStart.set(Provable.if(isFull, advancedStart, windowStart));
this.windowSize.set(Provable.if(isFull, windowSize, windowSize.add(1)));
```

There are two regimes. During the first 32 updates the bridge ever performs, the window is not yet full, `isFull` is false, and `windowStart` stays frozen. In that regime `noriMint` has a tolerance of about four `update` calls.

This is not how the bridge runs in normal operation. In steady state the window is full, so every `update` call evicts the oldest root and advances `windowStart`. A mint built while `windowStart` holds a given value carries a strict precondition pinning that value; the next update moves `windowStart`, so the mint is rejected after a single concurrent update, regardless of the five-slot action-state grace.

In other words, in normal operation a mint that has been built before but applied after the next update is invalidated, with no grace at all. Note that this is only about a specific `noriMint` proof being rejected after an update. The deposit root remains mintable for the full window of roughly 32 updates, so the user can always rebuild a fresh mint against the current `windowStart`; but each individual mint attempt only survives until the next update.

Impact

Nori's stated design goal is that a user can lock on Ethereum, leave, and return some time later (within a window of several hours) to claim their mint. The root-availability window of roughly 32 updates does support this in the sense that the deposit root remains mintable for that period. However, individual mint transactions only survive for as long as no later `update` was applied

on-chain. As a rough estimate, if it takes about two minutes from fetching current state to finishing a mint proof and it being applied on-chain, while the operator maintains a roughly 16-minute update cadence, then users will experience rejections of their `noriMint` transactions on the order of $2/16 = 12.5\%$ of attempts. If a rejection happens, they can continue to retry within the 32-update window.

The practical significance is therefore one of liveness and availability rather than soundness. A user who builds and sends promptly usually succeeds, but a meaningful share of attempts require a retry, which degrades the user experience and the economic viability of the bridge if users must repeatedly contend with mint rejections.

Recommendations

The underlying issue is that `noriMint` pins shared, operator-mutated state `windowStart` as a precondition, so it necessarily races every `update`. To avoid the described rejections, we recommend changing the design so that `noriMint` no longer depends on `windowStart`.

Alternatively, we recommend changing the documentation and user-facing messaging to distinguish the root-availability window (roughly 32 updates) from the per-attempt survival window (one update in steady state), and consider adding functionality to the user-facing tooling to handle mint rejections with repeat attempts, to avoid user surprise at a failure.

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit aed341b3 ↗ in the nori-bridge-sdk repository. Zellic did not review this change (see section 6.1. ↗).

3.10. [8ff57] Empty updates response causes `get_first_update` to panic

Target	bridge-head::nori::rpcs::consensus		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

The function `Client::get_first_update` fetches a single light client update for the current sync period and unconditionally indexes element 0 of the returned vector:

```
// nori-bridge-head: nori/src/rpcs/consensus/mod.rs
pub async fn get_first_update(&self) -> Result<Update<S>> {
    let period =
    calc_sync_period::<S>(self.get_current_finalizer_header_beacon_slot());

    // Handling the result and converting errors to anyhow::Error
    let updates_result = self
        .inner
        .rpc
        .get_updates(period, 1)
        .await
        .map_err(|e| Error::msg(e.to_string())); // Convert error to
    anyhow::Error

    match updates_result {
        Ok(mut updates) => Ok(updates.get_mut(0).unwrap().clone()), // Clone
    the updates if the result is Ok
        Err(e) => Err(e), // Propagate error if it's an Err
    }
}
```

The `.map_err(...)` only captures RPC errors. A successful HTTP response carrying an empty updates array (`[]`) is `Ok(vec![])`, which flows straight into `updates.get_mut(0).unwrap()` and panics.

In contrast, the function `prepare_consensus_proof_inputs` handles the possibility of empty updates with proper error handling instead of panicking:

```
// nori-bridge-head: nori/src/rpcs/consensus/mod.rs
let mut updates = client.get_updates().await?;
```

```
// Panic if our updates were empty (not sure how to deal with this yet)
if updates.is_empty() {
    return Err(anyhow::anyhow!("Error updates were missing 0th update."));
}
```

Impact

The function `get_first_update` is used in the cold start procedure, called via the inner call to `get_latest_finality_slot_and_store_hash` below:

```
// nori-bridge-head: nori/src/rpcs/consensus/mod.rs

/// Get the latest slot & store hash from the latest finality checkpoint.
pub async fn get_latest_finality_slot_and_store_hash(&self) -> Result<(u64,
    FixedBytes<32>)> {
    // This is used in cold start procedure which is a trusted operation
    (hence the principle trusted endpoint).
    // FIXME this bootstrap needs to be more strictly defined
    // but leaving this for now. We could always define a checkpoint file
    // to ourselves to avoid this being invoked.
    query_with_fallback(
        &self.principal_provider_url,
        &self.backup_providers_urls,
        |url| {
            async move { Client::<S,
    R>::get_latest_finality_slot_and_store_hash(&url).await }
                .boxed()
        },
        CONSENSUS_PROVIDER_TIMEOUT,
    )
    .await
}
```

An error from `get_first_update` bubbles up via the inner `get_latest_finality_slot_and_store_hash`, and then `query_with_fallback` will proceed trying backup provider urls, only erroring itself should all of them fail. Should all providers cause an error, the error returned by `get_latest_finality_slot_and_store_hash` will cause a panic in the top level caller, in `nori/bin/nori_bridge_head.rs`.

This intended fallback mechanism is however bypassed if `updates.get_mut(0).unwrap()` panics, and no backup providers are tried. This can affect the liveness of the cold start procedure.

Recommendations

Add an `is_empty()` check prior to indexing into the `updates` vector, and return an error if `updates` is empty.

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit 67e97e07 ↗ in the nori-bridge-head repository. Zellic did not review this change (see section 6.1. ↗).

3.11. [7f3a1] Mempool-visible `codeChallenge` allows griefing of `lockTokens` callers

Target	NoriTokenBridge (Ethereum)		
Category	Protocol Risks	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The `lockTokens` function of the Ethereum contract is used to lock ETH in the contract, bound to a `codeChallenge` which allows minting on the Mina side of the bridge. On the first call for a given `codeChallenge`, the zero-address check assigns the sender as the permanent depositor for that key, stored in `depositKeyToEthAddress`:

```
address linkedEthAddress = depositKeyToEthAddress[codeChallenge];
if (linkedEthAddress == address(0)) {
    // First deposit: bind Mina account deposit key to sender
    depositKeyToEthAddress[codeChallenge] = msg.sender;
} else {
    if (linkedEthAddress != msg.sender)
        revert MinaAccountLinkedToDifferentDepositor();
}
```

Because `codeChallenge` appears in calldata, it is visible in the mempool before it confirms. An observer can copy a pending `codeChallenge` and broadcast their own `lockTokens` call with a higher priority fee, causing their transaction to confirm first and binding their address to the `codeChallenge`. The honest depositor's transaction then reverts with `MinaAccountLinkedToDifferentDepositor`, and the depositor must generate a fresh `codeChallenge` and resubmit.

Impact

No depositor funds are at risk. The honest depositor's transaction reverts so that no ETH is transferred. The attacker, however, locks their own ETH under a `codeChallenge` they cannot redeem: `codeChallenge` is the hash of the SCRAM signature, and minting on Mina requires presenting that signature, which only the honest depositor holds. Thus the outcome will be that the honest depositor will have failed to lock their intended ETH, but will be able to mint the attacker's locked amount on the Mina side. The attack is economically self-destructive; each griefing attempt costs the attacker at least `MIN_LOCK_AMOUNT_WEI` (0.0001 ETH) with no possible return, and the victim can capture that amount.

Recommendations

We recommend removing the depositor binding entirely by eliminating the `depositKeyToEthAddress` mapping and the `MinaAccountLinkedToDifferentDepositor` check. This is safe because the unlock path in `unlockTokens` never consults `depositKeyToEthAddress`; the receiver and the amount to release are derived entirely from the Mina account proof. Without the binding check, an attacker who front-runs `lockTokens` with a copied `codeChallenge` no longer causes the honest depositor's transaction to revert: both transactions confirm, both contribute ETH to `lockedTokens[codeChallenge]`, and the honest depositor mints and later burns the combined amount on Mina as normal.

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit a5c75a3a ↗ in the nori-bridge-sdk repository. Zellic did not review this change (see section 6.1. ↗).

3.12. [1f602] Function `wordToBytes` does not guarantee canonicity for large `bytesPerWord`

Target	proof-conversion		
Category	Soundness	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The codebase contains two copies of `wordToBytes`, one in src/sha/sha_hash.ts and one in src/sha/utils.ts. Both build the same circuit for `Field -> UInt8[]`, decomposing a value into a little-endian byte array of length `bytesPerWord`:

```
// proof-conversion: src/sha/utils.ts

function wordToBytes(word: Field, bytesPerWord = 8): UInt8[] {
  let bytes = Provable.witness(Provable.Array(UInt8, bytesPerWord), () => {
    let w = word.toBigInt();
    return Array.from({ length: bytesPerWord }, (_, k) =>
      UInt8.from((w >> BigInt(8 * k)) & 0xffn)
    );
  });

  // check decomposition
  bytesToWord(bytes).assertEquals(word);

  return bytes;
}
```

Each witnessed byte is a `UInt8`, so `Provable.witness` range-checks it to $[0, 255]$ in-circuit (`UInt8.check` calls `RangeCheck.rangeCheck8`). The only other constraint is the reconstruction equality `bytesToWord(bytes).assertEquals(word)`, where:

$$\texttt{bytesToWord}(\texttt{bytes}) = \sum_{k=0}^{\texttt{bytesPerWord}-1} \texttt{bytes}[k] \cdot 2^{8k}$$

This equality is enforced over the field, that is, modulo p, the characteristic of `Field` (the Pallas base field), a 255-bit prime. The decomposition is therefore only guaranteed to be canonical if the full range of representable byte arrays cannot wrap around the modulus, that is when $2^{8 \cdot \texttt{bytesPerWord}} \leq p$, which for the Pallas base field means $\texttt{bytesPerWord} \leq 31$. The function does

not assert any bound on `bytesPerWord`, so it silently relies on every caller passing a small enough value.

If `bytesPerWord` is large enough that $2^{8 \cdot \texttt{bytesPerWord}} > p$ (i.e., `bytesPerWord` ≥ 32), the reconstruction constraint no longer pins down a unique byte array: multiple distinct `UInt8[]` values reconstruct to the same `word` modulo p, and a byte array may reconstruct to a `word` whose integer value differs from the array's integer value by a multiple of p. In that regime the returned bytes are not a sound big-integer decomposition of `word`.

Put differently, in this regime the circuit does not implement a function of its input: for a single `word` there are several distinct byte arrays that all satisfy the constraints, so the decomposition it computes is non-deterministic. Which of the valid outputs is returned is fixed only by the witness generator (the `Provable.witness` callback), and a malicious prover is free to satisfy the constraint system with any of the other valid byte arrays. Downstream logic that consumes the bytes and assumes they are the unique canonical little-endian representation of `word` therefore cannot rely on that being the case.

Impact

No soundness or completeness impact has been identified in the audited code. All current call sites in proof-conversion pass a `bytesPerWord` well within the safe range:

- src/sha/sha_hash.ts:100, src/plonk/fiat-shamir/index.ts:573, and src/blobstream/batcher.ts:264 call `wordToBytes(x.value, 4)` (4 bytes, 32 bits).
- src/blobstream/verify_blobstream.ts:33 calls `wordToBytes(num.toFields()[0])` with the default `bytesPerWord = 8` on a `UInt64` field, which is bounded to 64 bits.

In each case $2^{8 \cdot \texttt{bytesPerWord}} \leq 2^{64} \ll p$, so the decomposition is canonical and the missing bound has no effect today.

In nori-bridge-sdk, several calls use `bytesPerWord = 32`. Apart from tests, the usages of `wordToBytes` occur in:

- o1js-zk-utils/src/utils.ts and contracts/mina/src/scram.ts, where `bytesPerWord = 32`, but the circuit variables are immediately converted to a `BigInt` or string, with the circuit variables themselves not used further.
- o1js-zk-utils/src/types.ts, where values `1` and `31` are used for `bytesPerWord`.

While we are not aware of incorrect usage in the audited codebase, the correctness of the decomposition rests on an invariant that the function neither documents nor enforces. Future development could add callers that do not abide by the restriction, or switch to a prime that makes the current calls unsound.

Recommendations

Add an explicit assertion in both copies of `wordToBytes` that the requested width fits inside the field, so that an out-of-range `bytesPerWord` throws at circuit-construction time rather than silently

producing a non-canonical decomposition.

```
if (1n << BigInt(8 * bytesPerWord) > Field.ORDER) {
  throw new Error(
    `wordToBytes: bytesPerWord=${bytesPerWord} exceeds the field capacity`
  );
}
```

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commits 7d49b9be ↗ and b08dae95 ↗ in the proof-conversion repository. Zellic did not review these changes (see section 6.1. ↗).

3.13. [b165f] Usage of `Provable.switch` without an explicit range constraint on `shift_power`

Target	proof-conversion::groth::verifier, proof-conversion::plonk		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

Six call sites across the Groth16 and PLONK verifier paths use `Provable.switch` to select the shift factor applied to the Miller-loop accumulator. The intended domain of `shift_power` is $\{0, 1, 2\}$, corresponding to multiplying the accumulator by 1, ω_{27}, or ω_{27}^2 respectively. A representative example from `src/groth/verifier.ts`:

```
// proof-conversion: src/groth/verifier.ts

const shift = Provable.switch(
  [
    shift_power.equals(Field(0)),
    shift_power.equals(Field(1)),
    shift_power.equals(Field(2)),
  ],
  Fp12,
  [Fp12.one(), this.vk.w27, this.vk.w27_square]
);
f = f.mul(shift);

f.assert_equals(Fp12.one());
```

The same pattern appears in `src/groth/witness_tracker.ts`, `src/groth/recursion/zkp13.ts`, `src/plonk/mm_loop/multi_miller.ts`, `src/plonk/recursion/witness_tracker.ts`, and `src/plonk/recursion/zkp23.ts`. None of these sites constrain `shift_power` to $\{0, 1, 2\}$ before calling `Provable.switch`.

When `shift_power` is outside $\{0, 1, 2\}$, all three mask bits evaluate to `false`. The current o1js implementation of `Provable.switch` returns the zero element of the output type (`Fp12.zero()`) when all masks are false. The subsequent `f.assert_equals(Fp12.one())` then fails, so no valid proof can be generated with an out-of-range `shift_power`. There is no soundness gap under the current o1js behavior.

However, the o1js documentation for `Provable.switch` only describes its behavior if exactly one mask element is `true` and the others are `false`:

```
// o1js: src/lib/provable/provable.ts
  /**
   * Generalization of {@link Provable.if} for choosing between more than two
    different cases.
   * It takes a "mask", which is an array of `Bool`s that contains only one
    `true` element, a type/constructor, and an array of values of that type.
   * The result is that value which corresponds to the true element of the
    mask.
   * @example
   * ```ts
   * let x = Provable.switch([Bool(false), Bool(true)], Field, [Field(1),
    Field(2)]);
   * x.assertEquals(2);
   * ```
   */
  switch: switch_,
```

The all-false fallback returning zero is not documented as a guaranteed contract. A future o1js version could change this behavior and alter the verification logic without a changelog entry.

Impact

There is no soundness gap under the current implementations. The risk is that the six call sites rely on an undocumented implementation detail of `Provable.switch` in combination with the later assert in the callers that incidentally rules out the edge case. Future code changes could silently break the verification logic for both the Groth16 and PLONK verifiers.

Recommendations

Explicitly constrain that exactly one of the three mask `Bool`s is `true`, for example by constraining `shift_power` to be in $\{0, 1, 2\}$, or by constraining the sum of the mask `Bool`s to be one.

Alternatively, document the dependence on the current `Provable.switch` behavior as well as how this edge case is handled via the `f.assert_equals(Fp12.one());` constraint.

Remediation

This issue has been acknowledged by Nori, and a fix was implemented in commit df976e46 ↗.

3.14. [0dd9c] Function `ArrayListHasher.hash` does not enforce configured array length

Target	proof-conversion::array_list_hasher		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The class `ArrayListHasher` is a helper class with a fixed length `n` set to `ATE_LOOP_COUNT.length`. It occurs twice, once in `src/array_list_hasher.ts` (used by the Groth16 recursion programs), and similarly in `src/kzg/structs.ts` (used by the PLONK recursion programs):

```
// proof-conversion: src/array_list_hasher.ts
class ArrayListHasher {
  static n: number;

  static empty(): Field {
    const a = new Array(this.n).fill(Field(0n));
    return Poseidon.hashPacked(Provable.Array(Field, this.n), a);
  }

  static hash(arr: Array<Field>): Field {
    return Poseidon.hashPacked(Provable.Array(Field, this.n), arr);
  }
  // ...
}
ArrayListHasher.n = ATE_LOOP_COUNT.length;
```

The intent suggested by the structure is that `hash` computes the digest of an array of exactly `this.n` field elements. However, `this.n` does not actually constrain the computation. `Poseidon.hashPacked(type, value)` invokes `type.toInput(value)` and then packs the resulting `HashInput`:

```
// o1js: src/lib/provable/crypto/poseidon.ts

hashPacked<T>(type: WithProvable<Hashable<T>>, value: T) {
  let input = ProvableType.get(type).toInput(value);
  let packed = packToFields(input);
  return Poseidon.hash(packed);
},
```

The `toInput` derived for `Provable.Array(Field, n)` is (see `o1js/src/lib/provable/provable.ts`):

```
// o1js: src/lib/provable/provable.ts

function provableArray<A extends FlexibleProvableType<any>>(
  elementType: A,
  length: number
): InferredProvable<ToProvable<A>[]> {
  // ...
  return {
    // ...
    toInput(array) {
      // ...
      return array.reduce(
        (curr, value) => HashInput.append(curr, type.toInput(value)),
        HashInput.empty
      );
    }
    // ...
  } // ...
}
```

This is a fold over the actual JavaScript array, which does not use `length` (which here has the value `ArrayListHasher.n = ATE_LOOP_COUNT.length`). Hence, `ArrayListHasher.hash(arr)` returns `Poseidon.hash(arr)`, even if the length of `arr` differs from `this.n = ATE_LOOP_COUNT.length`.

The variant defined in `src/kzg/structs.ts` makes the same call but passes `ATE_LOOP_COUNT.length` directly rather than via `this.n`. The resulting behavior is identical.

Impact

In the audited code, every call site happens to pass an array whose length is already fixed to be `ATE_LOOP_COUNT.length`.

But future code changes could miss this and pass an array of a different length, and the `hash` method will not flag the mistake. This could lead to accidental generation of incorrect circuits, or harder-to-debug errors on witness generation.

Recommendations

Add an explicit length assertion at the start of `ArrayListHasher.hash`:

```
static hash(arr: Array<Field>): Field {
```

```
  if (arr.length !== this.n) {
    throw new Error(
      `ArrayListHasher.hash: expected array of length ${this.n}, got ${arr.
          length}`
    );
  }
  return Poseidon.hashPacked(Provable.Array(Field, this.n), arr);
}
```

Apply the same change to `src/kzg/structs.ts` and prefer reading the length from `this.n` rather than from the external `ATE_LOOP_COUNT.length`, so both copies have the same shape. Consider also collapsing the two definitions into a single shared module to avoid having to keep them in sync.

Remediation

This issue has been acknowledged by Nori, and a fix was implemented in commit b4ca3c3e ↗.

3.15. [10b08] Parallel implementations of shared utilities across Groth16 and PLONK components

Target	proof-conversion::array_list_hasher, proof-conversion::kzg::structs, proof-conversion::aux_witness, proof-conversion::plonk::aux_witness, proof-conversion::line_parser, proof-conversion::plonk::recursion::line_parser		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

Three utility types are independently reimplemented for the Groth16 and PLONK components, with no shared code and no cross-reference between them: `ArrayListHasher`, `AuXWitness`/`AuXWitnessType`, and `LineParser`. In each case the two copies implement the same logic, but live in unrelated directories with no mechanism to keep them in sync.

`ArrayListHasher`

The first copy lives in `src/array_list_hasher.ts` and is used by the Groth16 component (`src/groth`):

```
// proof-conversion: src/array_list_hasher.ts
class ArrayListHasher {
  static n: number;

  static empty(): Field {
    const a = new Array(this.n).fill(Field(0n));
    return Poseidon.hashPacked(Provable.Array(Field, this.n), a);
  }

  static hash(arr: Array<Field>): Field {
    return Poseidon.hashPacked(Provable.Array(Field, this.n), arr);
  }

  static open(
    lhs: Array<Field>,
    opening: Array<Fp12>,
    rhs: Array<Field>
  ): Field {
    const opening_hashes: Field[] = opening.map((x) =>
```

```
      Poseidon.hashPacked(Fp12, x)
    );
    let arr: Field[] = [];
    arr = arr.concat(lhs);
    arr = arr.concat(opening_hashes);
    arr = arr.concat(rhs);
    return this.hash(arr);
  }
}

ArrayListHasher.n = ATE_LOOP_COUNT.length;
```

The second copy lives in `src/kzg/structs.ts` and is used by the PLONK component (`src/plonk`):

```
// proof-conversion: src/kzg/structs.ts
class ArrayListHasher {
  static n: number;

  static empty(): Field {
    const a = new Array(this.n).fill(Field(0n));
    return Poseidon.hashPacked(Provable.Array(Field, ATE_LOOP_COUNT.length),
    a);
  }

  static hash(arr: Array<Field>): Field {
    return Poseidon.hashPacked(
      Provable.Array(Field, ATE_LOOP_COUNT.length),
      arr
    );
  }

  static open(
    lhs: Array<Field>,
    opening: Array<Fp12>,
    rhs: Array<Field>
  ): Field {
    const opening_hashes: Field[] = opening.map((x) =>
      Poseidon.hashPacked(Fp12, x)
    );
    let arr: Field[] = [];
    arr = arr.concat(lhs);
    arr = arr.concat(opening_hashes);
    arr = arr.concat(rhs);
    return this.hash(arr);
  }
}
```

```
ArrayListHasher.n = ATE_LOOP_COUNT.length;
```

Both classes set `ArrayListHasher.n = ATE_LOOP_COUNT.length` and produce the same output for the same input. The only difference is that the Groth16 version uses `this.n` inside `Provable.Array`, while the PLONK version hardcodes `ATE_LOOP_COUNT.length` directly. Since `this.n` is always assigned to `ATE_LOOP_COUNT.length`, the two are behaviorally identical.

AuXWitness / AuXWitnessType

The Groth16 copy lives in `src/aux_witness.ts`:

```
// proof-conversion: src/aux_witness.ts
export type AuXWitnessType = {
  c: Fp12Type;
  shift_power: string;
};

export class AuXWitness extends Struct({
  c: Fp12,
  shift_power: Field,
}) {
  static parse(path: string): AuXWitness {
    const data = fs.readFileSync(path, 'utf-8');
    const obj: AuXWitnessType = JSON.parse(data);

    return new AuXWitness({
      c: Fp12.loadFromJSON(obj.c),
      shift_power: Field.from(obj.shift_power),
    });
  }
}
```

The PLONK copy lives in `src/plonk/aux_witness.ts`:

```
// proof-conversion: src/plonk/aux_witness.ts
export type AuXWitnessType = {
  c: Fp12Type;
  shift_power: string;
};

export class AuXWitness extends Struct({
  c: Fp12,
  shift_power: Field,
```

```
}) {
  static loadFromPath(path: string): AuXWitness {
    const data = fs.readFileSync(path, 'utf-8');
    const obj: AuXWitnessType = JSON.parse(data);

    return new AuXWitness({
      c: Fp12.loadFromJSON(obj.c),
      shift_power: Field.from(obj.shift_power),
    });
  }

  static loadFromJSON(obj: AuXWitnessType): AuXWitness {
    return new AuXWitness({
      c: Fp12.loadFromJSON(obj.c),
      shift_power: Field.from(obj.shift_power),
    });
  }
}
```

The struct layout and constructor logic are identical. The only differences are that the Groth16 version names the path-loading method `parse`, while the PLONK version renames it `loadFromPath`, and the PLONK version adds a `loadFromJSON` method that accepts an already-parsed object.

LineParser

The Groth16 copy lives in `src/line_parser.ts`:

```
// proof-conversion: src/line_parser.ts
function ateCntSlice(from: number, to: number) {
  let line_cnt = 0;
  for (let i = from; i < to; i++) {
    if (ATE_LOOP_COUNT[i] == 0) {
      line_cnt += 1;
    } else {
      line_cnt += 2;
    }
  }
  return line_cnt;
}

class LineParser {
  static parse(from: number, to: number, lines: Array<G2Line>):
    Array<G2Line> {
    let start = ateCntSlice(1, from);
```

```
    let toSlice = ateCntSlice(from, to);
    return lines.slice(start, start + toSlice);
  }

  static frobenius_lines(lines: Array<G2Line>) {
    return lines.slice(-2);
  }
}
```

The PLONK copy lives in `src/plonk/recursion/line_parser.ts`:

```
// proof-conversion: src/plonk/recursion/line_parser.ts
function ateCntSlice(from: number, to: number) {
  let line_cnt = 0;
  for (let i = from; i < to; i++) {
    if (ATE_LOOP_COUNT[i] == 0) {
      line_cnt += 1;
    } else {
      line_cnt += 2;
    }
  }
  return line_cnt;
}

class LineParser {
  g2_lines: Array<G2Line>;
  tau_lines: Array<G2Line>;

  constructor(g2_lines: Array<G2Line>, tau_lines: Array<G2Line>) {
    this.g2_lines = g2_lines;
    this.tau_lines = tau_lines;
  }

  static init() {
    // loads g2_lines and tau_lines from precomputed JSON files
    ...
    return new LineParser(g2_lines, tau_lines);
  }

  parse_g2(from: number, to: number) {
    let start = ateCntSlice(1, from);
    let toSlice = ateCntSlice(from, to);
    return this.g2_lines.slice(start, start + toSlice);
  }

  parse_tau(from: number, to: number) {
```

```
    let start = ateCntSlice(1, from);
    let toSlice = ateCntSlice(from, to);
    return this.tau_lines.slice(start, start + toSlice);
  }

  frobenius_g2_lines() { return this.g2_lines.slice(-2); }
  frobenius_tau_lines() { return this.tau_lines.slice(-2); }
}
```

The `ateCntSlice` helper and the core slicing logic (`lines.slice(start, start + toSlice)`) are identical in both files. The structural difference is that the Groth16 version is stateless, receiving the line array as a parameter, while the PLONK version is a stateful wrapper that stores `g2_lines` and `tau_lines` on the instance.

Impact

In all three cases there is no technical reason that requires duplication, so the copies could be unified. During future development, the duplicated copies may silently diverge: a later developer might change one while assuming the behavior of the other, introducing bugs. Duplication also adds maintenance overhead, since any improvement to one copy (a bug fix, a performance change, better documentation) must be applied to the other by hand, or the other copy is left without it.

Recommendations

We recommend deduplicating all three cases. Note that for the copies in which interfaces have slight divergences, callers will need to be adjusted.

`ArrayListHasher`

Consolidate into a single shared implementation in `src/array_list_hasher.ts`, and update `src/kzg/structs.ts` to import from there instead of defining its own copy.

`AuXWitness`

Consolidate into `src/aux_witness.ts`. Rename `parse` to `loadFromPath` and add the `loadFromJSON` method so that both Groth16 and PLONK callers can use the shared implementation. The PLONK copy in `src/plonk/aux_witness.ts` can then simply import from `../aux_witness.ts`.

`LineParser`

Consolidate into a single shared implementation, deciding on either the stateful or stateless pattern.

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commits b4ca3c3e ↗ and 874d0457 ↗ in the proof-conversion repository. Zellic did not review these changes (see section 6.1. ↗).

3.16. [1a697] Function `Field.toBigInt` is a debug-only o1js operation but is used outside debug contexts

Target	proof-conversion		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The o1js `Field.toBigInt` method is documented in the library as a debug-only operation. Its docstring (`src/lib/provable/field.ts:155` in o1js) states:

> **Warning**: This operation does _not_ affect the circuit and can't be used to prove anything about the bigint representation of the {@link Field}. Use the operation only during debugging.

This finding concerns only calls of `Field.toBigInt` on o1js `Field` values, which are the values the debug-only warning applies to. The separate `ForeignField.toBigInt` method (`src/lib/provable/foreign-field.ts:181` in o1js), used on `FpC`/`FrC` values, is documented merely as "Convert this field element to a bigint" and carries no debug-only restriction.

The audited code does take into account that this function does not affect the circuit. However, the `Field.toBigInt` calls occur for proof creation, which is a non-debug usage, in the following places:

Inside the `Provable.witness` callback, with the result constrained back in-circuit

- `src/sha/sha_hash.ts:68` (`wordToBytes`, with `bytesToWord(bytes).assertEquals(word)`)
- `src/sha/utils.ts:43` (`wordToBytes`, with `bytesToWord(bytes).assertEquals(word)`)
- `src/blobstream/batcher.ts:104` (`fieldToHalfHash`, with `halfHashToField(nums).assertEquals(f)`)

Off-circuit accumulator cloning

- `src/kzg/structs.ts:23` — `Field.from(this.lines_hashes_digest.toBigInt())` inside `KzgState.deepClone()`

Off-circuit control flow

- `src/ec/g2.ts:43` — `if (eq.toBigInt() === 1n)` inside `G2Affine.add`
- `src/lines/index.ts:28` — `if (eq.toBigInt() === 1n)` inside `G2Line.fromPoints`

Impact

No current soundness or completeness impact has been identified in the audited code. With the current implementation of `Field.toBigInt`, using it outside of a prover block on a non-constant value would throw an error and therefore be noticed.

However, using a library function that is marked as debug-only for production code bears risks. Its implementation could change in the future: the library could alter the behavior in a way that goes unnoticed but breaks proof generation. The correctness of a debug-only function is likely a low priority for library maintainers and changes could be made with no security risk assessment.

Recommendations

Replace the `Field.toBigInt` calls in the non-debug code paths listed above with `Field.toValue`.

The method `Field.toValue` (`src/lib/provable/field.ts:945` in o1js) is the `Provable<Field>` interface method for converting a `Field` to its plain-JS (`bigint`) value, and its documentation carries no debug-only restriction, unlike `Field.toBigInt`. Using it therefore relies on a stable, documented API surface rather than on a function whose contract explicitly limits it to debugging. Note that `Field.toValue` currently delegates to `Field.toBigInt` internally, so its present runtime behavior — including throwing when applied to a non-constant value outside of a prover block — is identical; the improvement is in the contract being depended upon, not in the current behavior.

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit f01d3c9a ↗ in the proof-conversion repository. Zellic did not review this change (see section 6.1. ↗).

3.17. [a9dea] Undocumented value-dependence for circuit-generating functions

Target	proof-conversion		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

o1js circuits are constructed by executing the relevant JavaScript code once, at circuit compile time, and capturing the o1js operations that the code invokes as constraints. A JavaScript-level `if (cond) { A } else { B }` does **not** translate into a circuit that branches on `cond`. Only one of the two arms is actually traced, and the resulting constraint system encodes only the operations from that arm.

The audited code has helper functions that take parameters whose type is a circuit variable type, but whose assigned values are used to select between alternative subroutines via a JS-level `if`, and whose body invokes constraint-emitting o1js operations in each branch. The function signatures strongly suggest that the functions are usable for circuit generation for a variable value of those parameters, which is not the case.

Note that a circuit variable, even a `Bool` circuit variable, will be an object that, when used as `cond`, will be converted to `true` for deciding on the branch, and branch `A` will be taken, even if the assigned value of the circuit variable may be `false`. The helper functions discussed in this finding all explicitly extract the value from the circuit variable, and so exhibit assigned-value-dependence regarding the branch choice.

If such a helper is called from inside a ZkProgram method or from any other tracing context, the constraints it contributes depend on the value of the condition `cond` at that exact call: the helper resolves the `if` at compile time using the value of the condition during that call, and specializes the circuit to the corresponding branch. None of these helpers carry documentation reflecting this restriction.

We found the following instances of this undocumented pattern.

`G2Affine.add(rhs)` (`src/ec/g2.ts:39`).

The body computes

```
const eq = this.equals(rhs);
let lambda;
if (eq.toBigInt() === 1n) {
  lambda = this.computeLambdaSame();
} else {
  lambda = this.computeLambdaDiff(rhs);
```

```
}
```

The branch is taken based on the bigint value of an o1js `Field`, and the two branches introduce different constraints, so specializing to one of them at circuit compile time would yield an unsound circuit. However, in its implementation in the used library version, `toBigInt` would throw on a non-constant value outside of a prover block — see finding 3.16. ↗. Thus `G2Affine.add` could not be called for constraint-generation without throwing.

The function has no callers in the audited code.

`G2Line.fromPoints(lhs, rhs)` (`src/lines/index.ts:24`).

Same `if (eq.toBigInt() === 1n)` shape, selecting between `computeLambdaSame` and `computeLambdaDiff` as well. Called transitively only from `computeLineCoeffs`, which itself is only invoked off-circuit (`parseProof`, `GrothVk` setup, `WitnessTracker`, `precompute_lines`).

Impact

There is no impact in the audited code as all callers do not expect the functions to implement in-circuit branching. Additionally, all cases are combined with a call to `toBigInt`, which, in its implementation in the used library version, would throw on a non-constant value outside of a prover block.

Recommendations

For each of the functions, add a documentation comment at the function definition stating that the function's branch structure is resolved at circuit-construction time.

Add an explicit throw if a non-constant value is supplied to the argument that the branching depends on to not rely on the behavior of the debug-only `toBigInt` function.

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit f01d3c9a ↗ in the proof-conversion repository. Zellic did not review this change (see section 6.1. ↗).

3.18. [adcd3] Groth16 proof parser does not validate that `piN` keys are sequential

Target	proof-conversion::groth::proof		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The file `src/groth/proof.ts` defines the Groth16 proof struct, and a `parseProof` function that reads a proof from a JSON file:

```
// proof-conversion: src/groth/proof.ts
export function parseProof(vk: GrothVk, path: string) {
  const inputCount = detectInputCountFromProof(path);
  const ProofClass = createProofClass(inputCount);
  return ProofClass.parse(vk, path);
}
```

The function `detectInputCountFromProof` counts how many of the keys `pi1` through `pi6` are present in the JSON:

```
// proof-conversion: src/groth/proof.ts
export function detectInputCountFromProof(path: string): number {
  const json: O1jsProof = JSON.parse(fs.readFileSync(path, 'utf-8'));
  let count = 0;
  for (let i = 1; i <= 6; i++) {
    if (json[`pi${i}` as PiKey]) count++;
  }
  return count;
}
```

The returned number `inputCount` is passed to `createProofClass`, which returns a proof class whose `pis` field is typed as `Provable.Array(FrC.provable, inputCount)`, which expects exactly `inputCount` elements. The function `ProofClass.parse` fills `pis` by iterating `i` from 1 to `inputCount` and skipping any key that is absent:

```
// proof-conversion: src/groth/proof.ts
const ProofClass = class extends Struct({
  // ...
  pis: Provable.Array(FrC.provable, inputCount),
```

```
}) {
  static parse(vk: GrothVk, path: string) {
    const json: O1jsProof = JSON.parse(fs.readFileSync(path, 'utf-8'));

    const publicInputs: FrC[] = [];
    for (let i = 1; i <= inputCount; i++) {
      const key = `pi${i}` as PiKey;
      const val = json[key];
      if (val) {
        publicInputs.push(FrC.from(val));
      }
    }
    // ...
    return new ProofClass({
      // ...
      pis: publicInputs,
    });
  }
};
```

Both functions implicitly assume that the public inputs appear under contiguous keys `pi1`, `pi2`, ..., `pi{n}`. Neither function checks whether the present keys are actually sequential.

If the proof JSON has a gap — for example, `pi1`, `pi2`, `pi4`, `pi5` are present but `pi3` is absent:

```
{
  "pi1": "19350802088444617183621339156085479077",
  "pi2": "61803236023146647725736150410140474743",
  "pi4": "21562946873603930677336510313045680800 7",
  "pi5": "66557041833169831909454682372200415143 76"
}
```

then `detectInputCountFromProof` returns 4. `ProofClass.parse` then iterates `i = 1, 2, 3, 4`: it finds `pi1`, `pi2`, skips the absent `pi3`, and finds `pi4`, producing an array of 3 elements instead of the expected 4. When this malformed struct is later used, many code paths will throw with errors such as:

```
Error: Expected array of length 3, got 0
This shouldn't have happened and indicates an internal bug.
```

Impact

In practice, all sources that produce Groth16 proof JSON files that we checked, in particular the conversion step in `src/compute/plans/{snarkjs,sp1}/groth16.ts`, always emit sequential `pi1`

through `pi{n}` keys, so this issue cannot be triggered via those paths in normal operation. The concern is code maturity: if the keys are ever non-sequential, due to a future computing plan or a hand-crafted test input, the resulting error message gives no indication of the real cause.

Recommendations

Add a sequential-key check in `detectInputCountFromProof` or at the start of `ProofClass.parse`. After counting the present keys, verify that every key `pi1` through `pi{count}` is actually present in the JSON, and report a descriptive error if any is missing.

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit 4209fb0b ↗ in the proof-conversion repository. Zellic did not review this change (see section 6.1. ↗).

3.19. [af97e] Usage of deprecated `Gadgets.SHA256` API

Target	proof-conversion::plonk::fiat-shamir::Sp1PlonkFiatShamir, proof-conversion::plonk::piop::hash_fr		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The functions `gammaKzgDigest_part0` and `gammaKzgDigest_part1` in `Sp1PlonkFiatShamir` (`src/plonk/fiat-shamir/index.ts`) use low-level functions from `Gadgets.SHA256`. The two methods split the `gamma_kzg` hash computation into two parts, passing the intermediate SHA-256 compression state between them. They are called from two separate `ZkPrograms` (`zkp7` and `zkp8` in `src/plonk/recursion`), one for each part.

```
// proof-conversion: src/plonk/fiat-shamir/index.ts
gammaKzgDigest_part0(
  proof: Sp1PlonkProof,
  vk: Sp1PlonkVk,
  linearized_cm_x: FpC,
  linearized_cm_y: FpC,
  linearized_opening: FrC
) {
  // ...
  let H = Gadgets.SHA256.initialState;
  for (let i = 0; i < 11; i++) {
    const messageBlock = chunks.slice(16 * i, 16 * (i + 1));
    let W = Gadgets.SHA256.createMessageSchedule(messageBlock);
    H = Gadgets.SHA256.compression(H, W);
  }
  return H;
}

gammaKzgDigest_part1(proof: Sp1PlonkProof, H: UInt32[]) {
  // ...
  const messageBlock = chunks;
  let W = Gadgets.SHA256.createMessageSchedule(messageBlock);
  H = Gadgets.SHA256.compression(H, W);

  this.gamma_kzg_digest = Bytes.from(
    H.map((x) => wordToBytes(x.value, 4).reverse()).flat()
  );
```

```
}
```

The `Gadgets.SHA256` gadget is however a deprecated SHA-256 API:

```
// o1js: src/lib/provable/gadgets/gadgets.ts
/**
 * ...
 * @deprecated {@link SHA256} is deprecated in favor of {@link SHA2}, which
    supports more variants of the hash function.
 */
SHA256: SHA256,
```

Other functions in `src/plonk/piop/hash_fr.ts` and `src/plonk/fiat-shamir/index.ts` do not require low level interfaces, and use the function `Hash.SHA2_256.hash` for hashing. This is an alias for the `hash` function of the same deprecated `Gadgets.SHA256` gadget however:

```
// o1js: src/lib/provable/crypto/hash.ts

/**
 * The SHA2 hash function with an output length of 256 bits.
 */
SHA2_256: {
  /**
   * Hashes the given bytes using SHA2-256.
   *
   * This is an alias for `Gadgets.SHA256.hash(bytes)`.\
   * See {@link Gadgets.SHA256.hash} for details and usage examples.
   */
  hash: Gadgets.SHA256.hash,
},
```

Impact

The gadget `Gadgets.SHA256` has been deprecated since o1js v2.3.0 ↗ (released March 10, 2025), replaced by `Gadgets.SHA2`, which supports SHA-224, SHA-256, SHA-384, and SHA-512 under a single interface. We are not aware of any issues with the deprecated code. However, deprecated code may stop being maintained, so it may not be updated to stay correct as its dependencies evolve. As a deprecated gadget, `Gadgets.SHA256` might also be removed in a future o1js release, after which the code will fail to compile.

Recommendations

Replace `Gadgets.SHA256` with the equivalent `Gadgets.SHA2` calls in both `gammaKzgDigest_part0` and `gammaKzgDigest_part1`, and replace `Hash.SHA2_256` with `Hash.SHA2` elsewhere.

Deprecated	Current
`Gadgets.SHA256.initialState`	`Gadgets.SHA2.initialState(256)`
`Gadgets.SHA256.createMessageSchedule(block)`	`Gadgets.SHA2.messageSchedule(256, block)`
`Gadgets.SHA256.compression(H, W)`	`Gadgets.SHA2.compression(256, H, W)`
`Hash.SHA2_256.hash(m)`	`Hash.SHA2.hash(256, m)`

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit b2406ef6 ↗ in the proof-conversion repository. Zellic did not review this change (see section 6.1. ↗).

3.20. [d7ffa] Usage of `@internal` o1js APIs

Target	proof-conversion::towers::assertMul		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The file `src/towers/assert-mul.ts` defines a function `assertMul` as well as a class `UnreducedSum`, both used for foreign-field arithmetic constraints that appear throughout the tower arithmetic in the Groth16 and PLONK verifiers. They make use of `Gadgets.ForeignField.assertMul` as well as `Gadgets.ForeignField.Sum`, both of which are explicitly tagged `@internal` in o1js's source:

```
// proof-conversion: src/towers/assert-mul.ts
type ForeignFieldSum = ReturnType<typeof Gadgets.ForeignField.Sum>;

function assertMul(
  a: AlmostReducedSum | AlmostForeignField,
  b: AlmostReducedSum | AlmostForeignField,
  c: UnreducedSum | ForeignField
) {
  // ...
  let bF = b.value.finishForMulInput(a.modulus);
  let cF = c.value.finish(a.modulus);

  let allConstant =
    a.value.isConstant() &&
    Gadgets.Field3.isConstant(bF) &&
    Gadgets.Field3.isConstant(cF);
  // ...
  Gadgets.ForeignField.assertMul(a.value, bF, cF, a.modulus);
}

class UnreducedSum {
  value: ForeignFieldSum;
  // ...
  constructor(input: ForeignField) {
    this.value = Gadgets.ForeignField.Sum(input.value);
    // ...
  }
}
```

```
// o1js: src/lib/provable/gadgets/gadgets.ts

/**
 * @internal
 * ...
 */
assertMul(
  x: Field3 | ForeignFieldSum,
  y: Field3 | ForeignFieldSum,
  z: Field3 | ForeignFieldSum,
  f: bigint,
  message?: string
) {
  return ForeignField.assertMul(x, y, z, f, message);
},

// ...

/**
 * @internal
 *
 * Lazy sum of {@link Field3} elements, which can be used as input to
    {@link Gadgets.ForeignField.assertMul}.
 */
Sum(x: Field3) {
  return ForeignField.Sum(x);
},
```

The `@internal` tag in o1js signals that the API carries no stability contract: behavior, preconditions, and gate layout can change between releases without appearing in the public changelog.

Impact

We are not aware of any current vulnerability related to the usage of these `@internal` APIs. The risk with using internal functionality is forward-looking. A silent change to the gate layout or preconditions of `Gadgets.ForeignField.assertMul` or `Gadgets.ForeignField.Sum` in a future o1js release could alter the constraint system without causing a TypeScript-level test failure, potentially allowing previously invalid witnesses to satisfy the updated constraints.

We note that since we began our audit, the upstream o1js `main` branch has removed the `@internal` tags from both APIs, which signals that they are being moved toward a public API.

Recommendations

Follow the upstream stabilization of these APIs and migrate to a stable o1js release that exposes them as public once one is available; this also aligns with moving off the current pre-release build (see 4.1. ↗). Until then, treat `src/towers/assert-mul.ts` as a file that requires explicit care on any o1js upgrade: confirm that the behavior of `assertMul` and `Sum` is unchanged before accepting the new version.

Remediation

This issue has been acknowledged by Nori.

3.21. [b8891] Constructor `AffineCache` assigns `yp_prime` twice

Target	proof-conversion::lines::AffineCache		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The constructor of `AffineCache` (`src/lines/precompute.ts`) assigns `this.yp_prime` twice:

```
// proof-conversion: src/lines/precompute.ts

constructor(p: G1Affine) {
  this.xp_neg = p.x.neg().assertCanonical();
  this.yp_prime = p.y.inv().assertCanonical();
  this.yp_prime = Provable.witness(FpC.provable, () =>
    p.y.inv().assertCanonical()
  );
  this.yp_prime.mul(p.y).assertEquals(FpC.from(1n));
  this.xp_prime = this.xp_neg.mul(this.yp_prime).assertCanonical();
}
```

The first assignment, `this.yp_prime = p.y.inv().assertCanonical()`, is immediately overwritten by the second assignment from `Provable.witness(...)`. The snippets

```
this.yp_prime = p.y.inv().assertCanonical();
```

and

```
this.yp_prime = Provable.witness(FpC.provable, () =>
  p.y.inv().assertCanonical()
);
this.yp_prime.mul(p.y).assertEquals(FpC.from(1n));
```

are both intended to assign and constrain `this.yp_prime` to be the inverse of `p.y` in canonical form.

Impact

The constructed `AffineCache` value is the same with or without the redundant first assignment, so soundness and completeness are unaffected. The cost is an unnecessary `FpC.inv()` + `assertCanonical()` per `AffineCache` construction.

Recommendations

Delete one of the two ways to assign and constrain `this.yp_prime`. The second variant is slightly more efficient. Additionally, deleting the first variant more obviously retains the previous behavior. Thus we recommend deleting the first variant:

```
constructor(p: G1Affine) {
  this.xp_neg = p.x.neg().assertCanonical();
  this.yp_prime = p.y.inv().assertCanonical();
  this.yp_prime = Provable.witness(FpC.provable, () =>
    p.y.inv().assertCanonical()
  );
  this.yp_prime.mul(p.y).assertEquals(FpC.from(1n));
  this.xp_prime = this.xp_neg.mul(this.yp_prime).assertCanonical();
}
```

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit 8b6b0d40 ↗ in the proof-conversion repository. Zellic did not review this change (see section 6.1. ↗).

3.22. [e68c5] Function `G2Line.evaluate_g1` is dead code and does not implement the correct evaluation of the line function on G_1

Target	proof-conversion::lines		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The `G2Line` struct encodes a line on the twisted BN254 curve. To evaluate the corresponding line function on a point on G1, the twist-isomorphism must be taken into account, which is done correctly in the function `psi`. The same struct also exposes a method `evaluate_g1` that takes a `G1Affine` and returns an `Fp2`:

```
// proof-conversion: src/lines/index.ts

evaluate_g1(p: G1Affine): Fp2 {
  let t = this.lambda.mul_by_fp(p.x);
  t = t.neg();
  t = t.add(this.neg_mu);
  return t.add_fp(p.y);
}
```

This method does not implement a meaningful evaluation of the line function on G_1.

Impact

The function is not used in the audited code. Future code edits could mistakenly use this function to compute the evaluation of the G2-line function on G1, which would be incorrect.

Recommendations

Delete `G2Line.evaluate_g1`.

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit 47fcd6d4 ↗ in the proof-conversion repository. Zellic did not review this change (see section 6.1. ↗).

3.23. [e4e27] Function `prepare_consensus_mpt_proof_inputs` reconstructs `ExecutionHttpProxy` from env on every window

Target	bridge-head::nori::rpcs::consensus		
Category	Optimization	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The function `ConsensusHttpProxy::prepare_consensus_mpt_proof_inputs` constructs a fresh `ExecutionHttpProxy` via `try_from_env()` on every invocation:

```
// nori-bridge-head: nori/src/rpcs/consensus/mod.rs

// Get Execution Proxy (Note this is a bit messy to do this here now FIXME)
let validated_consensus_mpt_proof_input_with_window =
    ExecutionHttpProxy::<S>::try_from_env()
    .prepare_consensus_mpt_proof_inputs(
        input_slot,
        output_slot,
        finalized_input_block_number,
        finalized_output_block_number,
        validated_consensus_proof_inputs,
        expected_output_store_hash
    )
    .await?;
```

This happens at least once per proving window (the function is the per-window orchestrator invoked from the finality change detector).

Impact

This is a performance optimization, as the per-window reconstruction of the `ExecutionHttpProxy` is redundant and unnecessary.

Recommendations

Construct the `ExecutionHttpProxy` once and reuse it across windows.

Remediation

This issue has been acknowledged by Nori. Nori indicated that this finding was addressed in commit 784e70bd ↗ in the nori-bridge-head repository. Zellic did not review this change (see section 6.1. ↗).

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Use of a pre-release o1js build (3.0.0-mesa.698ca) across the Nori codebases

During the audit we found that o1js is pinned to the exact pre-release version `3.0.0-mesa.698ca` in the following packages:

- proof-conversion: package.json (@nori-zk/proof-conversion)
    - nori-bridge-sdk: cache-server/package.json (@nori-zk/cache-server)
    - nori-bridge-sdk: o1js-zk-utils/package.json (@nori-zk/o1js-zk-utils)
    - nori-bridge-sdk: minimal-client/package.json
    - nori-bridge-sdk: contracts/mina/package.json (@nori-zk/mina-token-bridge)

We investigated why this pre-release version was chosen. The changelogs of nori-bridge-head and nori-bridge-sdk show a deliberate transition toward the Mesa update of the Mina protocol, and the proof-conversion package was bumped to the same version to match.

We also examined what the `698ca` version is and confirmed that it is a pre-release build published on 2026-04-27. At the time of writing, no stable release of version 3 has been published.

Release date in npm	Version
2026-03-30	3.0.0-mesa.0
2026-04-02	3.0.0-mesa-native.0
2026-04-27	3.0.0-mesa.698ca
2026-05-04	3.0.0-mesa.89164
2026-05-04	3.0.0-mesa.1
2026-06-02	3.0.0-mesa.final

Because we confirmed the intent behind this choice, we did not classify it as an informational finding. We record it here as a discussion point instead.

That said, full documentation for this pre-release version has not been published, and the internals of o1js fall outside our audit scope. We did not check for behavioral differences that may exist

between version 2 and version 3. Instead, we reviewed whether Nori uses o1js correctly, based on the established usage of version 2.

As a general matter, we recommend using stable and audited dependencies. Once a stable release of o1js version 3 becomes available, we recommend updating to it.

Note that not all packages of the project use the above o1js version. For example nori-hash/scripts/package.json in the nori-bridge-head repository pins version 2.12.0 instead. The related code was not in scope for this audit, and we did not check whether usage of different versions across the projects constitutes a problem in this case. Generally we recommend using a consistent version for dependencies.

4.2. Unused functions and constants in proof-conversion

The proof-conversion codebase currently contains numerous unused functions and constants. Some are never called; others are exported but never imported.

Unused functions

- ec/g2.ts: `G2Affine.add`, `G2Affine.frobFromInputs`, `G2Affine.negFrobFromInputs`, `G2Affine.hom`
- groth/vk.ts: `GrothVk.ic1` - `GrothVk.ic6`
    - A comment claims these are kept for backward compatibility. But unlike `ic0`, none is used.
- lines/index.ts: `G2Line.evaluate_g1`
- plonk/proof.ts: `Sp1PlonkProof.deserialize`
- plonk/recursion/witness_tracker.ts: `WitnessTracker.fullGHashes`
- plonk/utils.ts: `unsafeHex2Bytes`, `numOfUin256s`
- towers/assert-mul.ts: `UnreducedSum.toBigint`
- towers/fp2.ts: `Fp2.loadFromJson`
- towers/fp12.ts: `Fp12.neg`, `Fp12.conjugate`, `Fp12.frobenius_pow_p_with_gammas`, `Fp12.frobenius_pow_p_squared_with_gammas`, `Fp12.frobenius_pow_p_cubed_with_gammas`, `Fp12.exp_e`, `Fp12.display`, `Fp12.pow`
    - `Fp12.pow` is transitively dead: its only caller, `Fp12.exp_e`, is itself unused.
- towers/fr.ts: `xorFr`

Unused constants

- towers/consts.ts: `P_MINUS_1_DIV_6`, `BETA`, `atc`

Being unused is not itself a bug. The real problem is that unused code is never run, so it is never checked against what it is meant to do. That lets it go silently wrong in two ways.

First, code that is correct today can become wrong later. When the spec or a live counterpart

changes, the unused copy is not updated. It quietly falls out of date, and small differences accumulate one by one. If the code is later used somewhere, problems can arise. For example, several functions in active use (`frobenius`, `frobenius_pow_p`, etc.) have matching unused counterparts that take separate inputs (`frobFromInputs`, `frobenius_pow_p_with_gammas`, etc.). If the active ones are changed and the unused ones are not, anyone who later uses the unused version gets a stale result.

Second, code can be wrong from the start and never get caught, because nothing ever runs it. `G2Line.evaluate_g1` is such a case: its name promises one thing while its body does another (see 3.22. ↗). The mismatch has gone unnoticed only because the function is dead. A developer who later trusts the name will get a wrong result.

Some of these, such as `Fp12.neg`, `UnreducedSum.toBigint`, and the constants in `towers/consts.ts`, may look general and worth keeping for later reuse. But the same reasoning applies. While they sit unused, no one confirms they are still correct, and a future caller may run into a hidden bug. The safer course is to remove them now and reimplement them only when a real use appears.

4.3. Field element `w27` lacks a canonical source and documentation of its derivation

The value ω_{27} (referred to as `w27` throughout the proof-conversion codebase) is a fixed 27th root of unity in $\mathbb{F}_{p^{12}}$ required by the pairing optimization described in "On Proving Pairings" (2024/640) ↗. The value must satisfy $\omega_{27}^{27} = 1$ and $\omega_{27}^{9} \neq 1$, that is, it must have multiplicative order exactly 27 in $\mathbb{F}_{p^{12}}^{\times}$. The paper treats this as an arbitrary fixed input and specifies no concrete value, so an implementation must fix one.

The codebase uses the following concrete element as `w27`. Using the BN254 tower decomposition $\mathbb{F}_{p^{12}} = \mathbb{F}_{p^6}[w]/(w^2 - v), \mathbb{F}_{p^6} = \mathbb{F}_{p^2}[v]/(v^3 - (9 + u)), \mathbb{F}_{p^2} = \mathbb{F}_p[u]/(u^2 + 1)$:

$$\omega_{27} = (g_{20} + g_{21} \cdot u) \cdot v^2 \in \mathbb{F}_{p^{12}},$$
$$g_{20} = 8204864362109909869166472767738877274689483185363591877943943203703805152849,$$
$$g_{21} = 17912368812864921115467448876996876278487602260484145953989158612875588124088$$

This value is hardcoded in six independent locations across the codebase:

- pairing-utils/src/bin/convert_from_snarkjs.rs: `create_default_w27`
- pairing-utils/src/bin/convert_from_sp1_groth16.rs: `create_default_w27`
- pairing-utils/src/o1js.rs: `TryFrom<&SnarkjsVK> for O1jsVK`
- pairing-utils/src/o1js.rs: `TryFrom<&ark_groth16::VerifyingKey<Bn254>> for O1jsVK`
- src/plonk/helpers.ts: `make_w27`, used globally across the PLONK component
- src/plonk/mm_loop/helpers.ts: `make_w27`, only used in a test

In contrast, `compute_aux_witness` in pairing-utils/src/kzg.rs, which computes the auxiliary witnesses `c` and `shift_power` following the paper, obtains `w27` by calling `get_shift_factor()` rather than using a hardcoded constant:

```
// proof-conversion: pairing-utils/src/kzg.rs

fn get_shift_factor() -> Fq12 {
    let rng = &mut StdRng::seed_from_u64(123u64);
    sample_27th_root_of_unity(rng)
}
```

```
// proof-conversion: pairing-utils/src/utils.rs

// K = (p^12 - 1)/27
pub(crate) fn sample_27th_root_of_unity<R: RngCore>(rng: &mut R) -> Fq12 {
    let one = Fq12::one();

    let pow_9 = |x: Fq12| -> bool { x.pow(&[9, 0, 0, 0]) != one };

    loop {
        let x = Fq12::rand(rng);
        let w = exp(x, &K);
        if w != one && pow_9(w) {
            return w;
        }
    }
}
```

We verified directly that `get_shift_factor()` returns the same value as every hardcoded instance.

We identified one concern: the hardcoded `w27` value has no explanation of where it originates. Additionally, because this value is hardcoded, inconsistencies may arise during code modification. The `make_w27` function in src/plonk/helpers.ts carries no comment at all, and the Rust sites reference the paper as if the value were taken from it, even though the paper provides no specific numeric value:

```
// proof-conversion: pairing-utils/src/o1js.rs

// Hardcoded w27 (used for pairing optimizations)
// https://eprint.iacr.org/2024/640
```

Therefore, the codebase would benefit from consolidating `w27` to a single source of truth and documenting its derivation there. Since a mismatch in `w27` would clearly fail verification, this is not classified as a finding.

4.4. Difference in the KZG folding randomizer derivation between proof-conversion and the SP1 Rust verifier

Background

The final step of PLONK verification folds two KZG openings into one pairing check. It requires a folding randomizer, and we write this value as `u` below. In proof-conversion, `squeezeRandomForKzg` in `src/plonk/fiat-shamir/index.ts` produces it.

The SP1 v6.1.0 Rust verifier (`sp1/crates/verifier/src/plonk` ↗) was the reference given at the start of this audit. We compared `squeezeRandomForKzg` against it and found that the two derive `u` in different ways. We reported this difference to the client.

The client replied that proof-conversion matches the SP1 v6.1.0 Solidity verifier (`PlonkVerifier.sol#L534-L549` ↗, `#L811-L845` ↗), and also the upstream gnark template (`solidity.go#L823-L841` ↗). Both are established references, so proof-conversion's derivation is well-founded. The difference can also be seen as one between SP1's own Rust and Solidity verifiers, and it prompted us to look closer. We checked whether the difference affects the soundness of either derivation.

The discrepancy

The value `u` is the randomizer that folds the two KZG openings. We worked out how each verifier computes `u`, shown as the expressions below. The values are named as in the proof-conversion codebase.

The SP1 Rust verifier computes:

```
u = SHA256("u" || zeta_digest.bytes || gamma_kzg || cm || proof.grand_product
    || proof.batch_opening_at_zeta || proof.batch_opening_at_zeta_omega)
```

proof-conversion computes:

```
u = SHA256(cm || proof.batch_opening_at_zeta || proof.grand_product ||
    proof.batch_opening_at_zeta_omega || zeta || gamma_kzg)
```

Both hash the same set of values, with two exceptions. The Rust verifier prepends `"u"`. The Rust verifier hashes `zeta_digest.bytes`, while proof-conversion hashes `zeta`. The shared values also appear in a different order.

These values are the inputs to the two KZG opening checks that the fold combines. One opening is at `zeta`, the other at `zeta * omega`. Verifying an opening at a point involves the commitment, the claimed evaluation at that point, and the quotient proof. The claimed evaluations are not hashed

on their own. They enter through `gamma_kzg`, the batching randomizer.

- The check at `zeta` involves the commitment `cm`, the quotient proof `proof.batch_opening_at_zeta`, and `gamma_kzg`. `gamma_kzg` is hashed from the claimed evaluations of all openings folded at `zeta`.
- The check at `zeta * omega` involves the commitment `proof.grand_product`, the quotient proof `proof.batch_opening_at_zeta_omega`, and `gamma_kzg`. `gamma_kzg` is also hashed from the claimed evaluation `proof.grand_product_at_omega_zeta`.

The hash also carries `zeta`. `zeta` is the evaluation point of the first opening, a BN254 scalar in $\mathbb{F}_r$. It is the raw 32-byte SHA-256 output `zeta_digest.bytes` taken modulo r. The second point `zeta * omega` is `zeta` times the fixed root of unity `omega` from the verifying key, so committing to `zeta` alone fixes both points.

For the fold to be sound, the derivation of `u` must commit to the values the two opening checks involve. In both implementations, `u` is bound to these values, either directly or through `gamma_kzg`. So an attacker must fix every value before `u` is known, in both. In this sense, the input ordering does not affect soundness.

Next is the `"u"` tag prepended in the Rust verifier. The Rust verifier reuses its Fiat-Shamir transcript struct to produce `u`, so the hash input follows the transcript layout. `"u"` is the domain separator that the struct assigns to `u`. proof-conversion builds the input directly, without that struct, so it carries no such tag. Since the tag is a fixed constant, it does not change what `u` binds, so it does not affect soundness.

One difference still needs analysis: whether deriving `u` from `zeta` instead of `zeta_digest.bytes` affects soundness.

Effect of `zeta` versus `zeta_digest.bytes`

The fold combines the two openings with coefficients `1` and `u`, so the check is degree 1 in `u`. If either opening is invalid, then by the Schwartz-Zippel lemma a random `u` passes it with probability at most $1/r \approx 1/2^{254}$.

The Rust verifier hashes `zeta_digest.bytes` instead of the reduced `zeta`. Reduction modulo r is many-to-one: each `zeta` has five or six digests reducing to it, since $2^{256}/r \approx 5.29$. These digests give the same check but a different `u`. An attacker who could find several `zeta_digest.bytes` reducing to one `zeta` would then get five or six tries at the single root, raising the error to between $5/r$ and $6/r$. But finding such values is a collision on a roughly 254-bit value, which takes about 2^{127} hashes by the birthday bound, so this does not happen practically. Also, the issue does not apply to proof-conversion at all, since it hashes the reduced `zeta`, where one `zeta` gives exactly one `u`.

Conclusion

We classify this as a discussion item, not a finding. Both derivations bind `u` to the values the two checks involve, so both are sound. The `zeta_digest.bytes` concern is specific to the Rust verifier

and rests on `zeta` collisions that are infeasible to find. proof-conversion uses the reduced `zeta`, so it is not affected.

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the smart contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. Module: MinaStateSettlement.sol

Function: `updateChain(bytes32 proofCommitment, bytes32 provingSystemAuxDataCommitment, bytes20 proofGeneratorAddr, bytes32 batchMerkleRoot, bytes memory merkleProof, uint256 verificationDataBatchIndex, bytes memory pubInput, address batcherPaymentService)`

Verifies Mina state-proof inclusion via Aligned Layer and records the finalized candidate-chain ledger hash (index 0 of `candidate_chain_ledger_hashes`) in `verifiedLedgerHashes`. The function is permissionless; safety comes from proving-system ID checks, network-flag checks, and Aligned batch-inclusion verification.

Inputs

- `proofCommitment`
  - **Control**: Fully controlled.
  - **Constraints**: Must match the verification-data commitment included in the referenced Aligned batch leaf.
  - **Impact**: If mismatched, `aligned.verifyBatchInclusion(...)` returns false/reverts and `updateChain` fails.
- `provingSystemAuxDataCommitment`
  - **Control**: Fully controlled.
  - **Constraints**: Must equal `PROVING_SYSTEM_ID_COMM` for Mina state proofs.
  - **Impact**: Wrong value reverts immediately with `MinaProvingSystemIdIsNotValid`.
- `proofGeneratorAddr`
  - **Control**: Fully controlled.
  - **Constraints**: Must match the proof-generator address committed in the Aligned verification data.
  - **Impact**: Inconsistency causes verification failure.
- `batchMerkleRoot`
  - **Control**: Fully controlled.
  - **Constraints**: Must be the Merkle root of the batch containing the target

verification data.
    - **Impact**: Invalid root causes inclusion verification failure.
- `merkleProof`
    - **Control**: Caller-supplied Merkle branch bytes.
    - **Constraints**: Must prove the verification-data leaf at `verificationDataBatchIndex` under `batchMerkleRoot`.
    - **Impact**: Invalid proof prevents successful verification and state update.
- `verificationDataBatchIndex`
    - **Control**: Caller-supplied index.
    - **Constraints**: Must be the exact leaf position used by `merkleProof`.
    - **Impact**: Wrong index makes proof invalid.
- `pubInput`
    - **Control**: Caller-supplied serialized Mina state proof public input.
    - **Constraints**: `pubInput[0]` must match `devnetFlag`; full bytes must match the commitment verified by Aligned (`keccak256(pubInput)`); layout must contain the finalized ledger hash at the expected offset (`0x241` in memory).
    - **Impact**: Determines both network acceptance and which ledger hash is marked verified; malformed/wrong data causes network-check revert or inclusion-check failure.
- `batcherPaymentService`
    - **Control**: Fully controlled.
    - **Constraints**: Must match the payment-service address committed in the verification data.
    - **Impact**: Mismatch causes verification failure.

Branches and code coverage

Intended branches

- Successful path stores finalized ledger hash and emits `LedgerHashVerified`.
    - ☐ Test coverage
- Idempotent behavior when submitting the same proof twice (mapping remains `true`).
    - ☐ Test coverage

Negative behavior

- Wrong proving-system commitment reverts with `MinaProvingSystemIdIsNotValid`.
    - ☐ Negative test
- Wrong devnet/mainnet flag in `pubInput[0]` reverts with `MinaNetworkIsWrong`.

- ☐ Negative test

- `aligned.verifyBatchInclusion == false` reverts with `NewStateIsNotValid`.
  - ☐ Negative test

Function call analysis

- aligned.verifyBatchInclusion(proofCommitment, pubInputCommitment, provingSystemAuxDataCommitment, proofGeneratorAddr, batchMerkleRoot, merkleProof, verificationDataBatchIndex, batcherPaymentService)
  - **What is controllable?** All arguments controlled by the caller, except `pubInputCommitment`, which is computed as `keccak256(pubInput)`. The `pubInput` is also specified by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** `false` result triggers `NewStateIsNotValid`, `true` allows extracting/storing finalized ledger hash.
  - **What happens if it reverts, reenters or does other unusual control flow?** Function is external and permissionless, but this call sends no ETH and `updateChain` has no callbacks, so reentrancy surface is minimal; unusual control flow is primarily verifier revert.

5.2. Module: NoriTokenBridge.sol

Function: `lockTokens(uint256 codeChallenge)`

Locks ETH for a Mina deposit key identified by `codeChallenge` (hash of the Mina signature). Requires the bridge to be configured, `msg.value` to be at least `MIN_LOCK_AMOUNT_WEI`, and `msg.value` to be an exact multiple of `WEI_PER_BRIDGE_UNIT`; otherwise it reverts. Converts `msg.value` to bridge units, deducts `lockFeeRate` (with `MIN_FEE_BU` floor when fee is enabled), and enforces `totalLockedBU + netBU <= MAX_MAGNITUDE`. This function binds each `codeChallenge` to the first depositor address in `depositKeyToEthAddress`, as a result, the subsequent deposits for the same challenge are only accepted from that same address. Additionally, `lockedTokens`, `totalLockedBU`, and `accumulatedFees` are updated and `TokensLocked(msg.sender, codeChallenge, netWei, feeWei)` is emitted.

Inputs

- `codeChallenge`
  - **Control**: any caller can submit a code challenge
  - **Constraints**: `depositKeyToEthAddress` for the code challenge should be empty, otherwise a caller should match the existing depositor address.
  - **Impact**: the code challenge is bound to the depositor address, and the locked amount is added to the `lockedTokens` mapping for the code

challenge.

Branches and code coverage

Intended branches

- Successful lock with aligned value at zero fee stores bridge units.
  - ☑ Test coverage
- Successful lock with nonzero fee stores net BU and accumulates fees.
  - ☑ Test coverage
- Exact minimum lock amount succeeds.
  - ☑ Test coverage
- Repeated lock by same depositor for same `codeChallenge` accumulates.
  - ☑ Test coverage
- First lock binds `codeChallenge` to depositor address.
  - ☑ Test coverage
- `TokensLocked` event emitted with expected net/fee wei amounts (zero and nonzero fee cases).
  - ☑ Test coverage

Negative behavior

- Below-minimum amount reverts with `BelowMinLockAmount` (including dust case).
  - ☑ Negative test
- Non-bridge-unit-aligned value reverts with `InvalidBridgeUnitMultiple`.
  - ☑ Negative test
- Different depositor using existing `codeChallenge` reverts with `MinaAccountLinkedToDifferentDepositor`.
  - ☑ Negative test
- Overflow beyond `MAX_MAGNITUDE` reverts with `TotalLockedOverflow`.
  - ☑ Negative test
- Lock while unconfigured reverts with `AlignedContractsNotConfigured`.
  - ☐ Negative test

Function: `setAlignedContracts(address _stateSettlementAddr, address _accountValidationAddr)`

Allows the bridge operator to rotate the `_stateSettlementAddr` and `_accountValidationAddr` used by `unlockTokens`. `_stateSettlementAddr` is stored as `stateSettlement` and is queried via `isLedgerVerified` to ensure the submitted Mina account proof references a finalized ledger hash accepted by state proofs. `_accountValidationAddr` is stored as `accountValidation` and is called via `validateAccount` to confirm Aligned batch inclusion and Mina account-proof validity before decoding account data and releasing ETH.

Inputs

- `_stateSettlementAddr`
  - **Control**: Caller-controlled. But the caller is the bridge operator, which is trusted.
  - **Constraints**: Must be a nonzero address.
  - **Impact**: This address is used to verify the ledger hash.
- `_accountValidationAddr`
  - **Control**: Caller-controlled. But the caller is the bridge operator, which is trusted.
  - **Constraints**: Must be a nonzero address.
  - **Impact**: This address is used to validate the account proof.

Branches and code coverage

Intended branches

- Operator can set aligned contract addresses and `StateSettlementSet` / `AccountValidationSet` events are emitted.
  - ☑ Test coverage
- Assert storage pointers (`stateSettlement`, `accountValidation`) are updated, not only events.
  - ☐ Test coverage

Negative behavior

- Non-operator caller reverts with `NotBridgeOperator`.
  - ☑ Negative test
- Zero state-settlement address reverts with `ZeroAddress`.
  - ☑ Negative test
- Zero account-validation address reverts with `ZeroAddress`.

☑ Negative test

Function: `unlockTokens(bytes32 proofCommitment, bytes32 provingSystemAuxDataCommitment, bytes20 proofGeneratorAddr, bytes32 batchMerkleRoot, bytes memory merkleProof, uint256 verificationDataBatchIndex, bytes calldata pubInput, address batcherPaymentService)`

Unlocks ETH to the Mina-designated receiver after verifying the submitted account proof against a previously verified ledger hash and the configured zkApp identity constraints. The function derives `tokensToUnlock` from the Mina burn-counter delta, updates local unlock/locked accounting, deducts `unlockFeeRate` (with `MIN_FEE_BU` floor when enabled), and transfers net ETH.

Inputs

- `proofCommitment`
  - **Control**: Fully controlled, forwarded in `AlignedArgs` to `accountValidation.validateAccount(args)`.
  - **Constraints**: Must match the commitment used in the Aligned batch for the provided account proof.
  - **Impact**: Wrong value makes verification fail, so unlock does not proceed.
- `provingSystemAuxDataCommitment`
  - **Control**: Fully controlled, forwarded in `AlignedArgs` to `accountValidation.validateAccount(args)`.
  - **Constraints**: Must match the expected proving-system commitment checked inside `MinaAccountValidation`.
  - **Impact**: Wrong value makes verification fail, so unlock does not proceed.
- `proofGeneratorAddr`
  - **Control**: Fully controlled, forwarded in `AlignedArgs` to `accountValidation.validateAccount(args)`.
  - **Constraints**: Must match the generator identity committed in the verified batch data.
  - **Impact**: Wrong value makes verification fail, so unlock does not proceed.
- `batchMerkleRoot`
  - **Control**: Fully controlled, batch root for inclusion verification.
  - **Constraints**: Must correspond to the batch that contains the referenced verification data leaf.
  - **Impact**: Invalid root causes inclusion verification failure and unlock revert.
- `merkleProof`
  - **Control**: Fully controlled, Merkle branch bytes.

    - **Constraints**: Must prove membership of the verification data in `batchMerkleRoot` at `verificationDataBatchIndex`.
    - **Impact**: Invalid proof prevents account validation.
- `verificationDataBatchIndex`
    - **Control**: Caller chooses the leaf index used for Merkle inclusion.
    - **Constraints**: Must match the actual position of the verification data in the batch tree.
    - **Impact**: Wrong index invalidates inclusion verification and blocks unlock.
- `pubInput`
    - **Control**: Fully controlled.
    - **Constraints**: Must encode ledger hash at bytes `[0:32]` and a decodable `Account` payload at `[40:]`
    - **Impact**: Drives ledger gating, account decoding, unlock amount calculation, receiver address.
- `batcherPaymentService`
    - **Control**: Fully controlled, forwarded in `AlignedArgs` to `accountValidation.validateAccount(args)`.
    - **Constraints**: Must be the same payment-service address committed in the verified batch data.
    - **Impact**: Wrong value makes verification fail, so unlock does not proceed.

Branches and code coverage

Intended branches

- Full happy path with mocked `stateSettlement` and `accountValidation`, correct zkApp key/token checks, fee deduction, ETH payout, and `TokensUnlocked` emission.
    - ☐ Test coverage
- Unlock with `unlockFeeRate == 0` and with nonzero `unlockFeeRate` including min-fee rounding branch.
    - ☐ Test coverage

Negative behavior

- Invalid ledger -> `InvalidLedger`.
    - ☐ Negative test
- Invalid account proof -> `InvalidZkappAccount`.
    - ☐ Negative test
- Wrong zkApp verification key -> `IncorrectZkappVerificationKey`.
    - ☐ Negative test

- Wrong tokenId -> `IncorrectTokenHolderAccount`.
  - ☐ Negative test
- Burn counter not increased -> `BurnCounterNotIncreased`.
  - ☐ Negative test
- `tokensToUnlock > totalLockedBU` and `tokensToUnlock <= feeBU` -> `InvalidUnlockAmount`.
  - ☐ Negative test
- Zero receiver and failed ETH transfer -> `ZeroAddress` / `EthTransferFailed`.
  - ☐ Negative test
- Reentrancy guard behavior on payout receiver callback.
  - ☐ Negative test

Function call analysis

- stateSettlement.isLedgerVerified(ledgerHash)
  - **What is controllable?** `ledgerHash`
  - **If the return value is controllable, how is it used and how can it go wrong?** Boolean gates the entire unlock path. A false value reverts with `InvalidLedger`.
  - **What happens if it reverts, reenters or does other unusual control flow?** No problems here.
- accountValidation.validateAccount(args)
  - **What is controllable?** All proof arguments are controlled by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** False triggers `InvalidZkappAccount`.
  - **What happens if it reverts, reenters or does other unusual control flow?** No problems here.
- payable(receiver).call{value: netWei}('')
  - **What is controllable?** `receiver` is derived from `account.zkapp.appState[3]` in proof-bound `pubInput`; `netWei` is derived from burn delta and fee configuration.
  - **If the return value is controllable, how is it used and how can it go wrong?** If return value is false, function reverts with `EthTransferFailed`.
  - **What happens if it reverts, reenters or does other unusual control flow?** Receiver fallback can execute arbitrary logic and attempt reentrancy; `nonReentrant` prevents re-entering `unlockTokens`/`withdrawFees`. Zero-address receiver is pre-checked and reverts with `ZeroAddress`.

6. Assessment Results

During our assessment on the scoped Nori Bridge circuits, we discovered 23 findings. Three critical issues were found. Three were of high impact, one was of medium impact, four were of low impact, and the remaining findings were informational in nature.

Based on the scope of findings identified during this engagement, we recommend a comprehensive reassessment of the codebase following the completion of remediation efforts. Our review uncovered a number of serious findings, some of which will require, or are already planned to require, significant design changes to remediate. For example, finding 3.6. ↗ will likely require a substantial redesign of how deposits are locked and accounted for in the Ethereum smart contract, together with corresponding changes to the SP1 guest program, so that the guest program can enforce eventual inclusion of deposits in the submitted order without introducing new bricking or denial-of-service avenues. Based on the results of our review, and given that this time-boxed engagement was performed on a best-effort basis (see 1.3. ↗) and was therefore limited in depth and coverage, we believe it is likely that additional issues remain undiscovered. Additionally, the remediation efforts themselves will introduce changes that may create new vulnerabilities, both independently and in combination with changes made as part of other remediations.

A follow-up assessment would provide greater confidence in the overall security posture of the protocol and help ensure that remediation efforts have not introduced new issues.

To maximize the value of a follow-up engagement, we recommend not only remediating the findings with immediate impact, but also addressing the code maturity observations identified during our review, such as code duplication and dead code. We further recommend improving the documentation of the codebase by ensuring that it remains up to date following changes, and by adding comprehensive but concise documentation describing the overall interplay between components and the trust assumptions made regarding external components and roles.

As a longer-term investment in code quality, we encourage continuing to strengthen security-focused development workflows, including:

- Comprehensive test coverage with end-to-end tests that exercise critical paths and edge cases
- Clear documentation of security assumptions, roles in the system, how the components connect[2], invariants, and known limitations
- Structured code review processes that explicitly consider security implications
- Removing code that becomes dead and preventing code duplication

A reassessment following these improvements would allow for deeper analysis and provide greater assurance for production deployment.

[2] For example, how the verification key will that the smart contract(s) will be configured with will be generated.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

In addition, for a number of findings, Nori provided commits that Nori states address the finding, but which Zellic did not review as part of this engagement. Where this is the case, the Remediation section of the finding states so explicitly. Zellic makes no representation as to whether these commits correctly or completely address the finding, or whether they introduce new issues. Such findings should be treated as acknowledged by Nori rather than as fixed, as Zellic has not verified the changes.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.